# Panel 9 (formerly Mobile Browser) — Architecture Guide

*Audience: another bot/engineer who must understand, modify, rebuild and re-publish this app. App-store id: `panel-9` (current 2.3.9). The old store id `mobile-browser` (≤1.1.x installs) is kept only as a hidden "bridge" so old installs upgrade to Panel 9 — see §7. Project folder: `/workspace/mobile-browser`.*

## 1. Overview

Panel 9 is an **Electron browser for Windows x64** (portable exe + NSIS installer) with Chrome-style **profiles**, **unlimited Google accounts per profile** (isolated "account containers"), private windows, history, downloads, favourites/bookmarks bar, a built-in home page, a settings page, and the original per-tab **desktop/mobile device emulation** (phone frame, UA/client-hints/touch/viewport spoofing from JSON device profiles). It updates itself from the Oluchi App Store, passes Google's embedded-browser check for sign-in (§9), and runs **Chrome-style extensions** distributed through the same store (§10).

## 2. Tech stack

Electron ^45 (Chromium engine) · plain CommonJS JavaScript (no bundler, no framework, no TypeScript) · UI = static HTML/CSS/JS rendered in `WebContentsView`s · electron-builder ^26 (win `portable` + `nsis`) · tests are plain scripts run under `xvfb-run electron`. Pure-Node modules (`store.js`, `profiles.js`, `updater.js`, `accounts.js` parsing, `migrate.js`) deliberately avoid importing Electron so they are unit-testable.

## 3. Folder map

```
package.json            version (2.3.6), scripts, electron-builder config (appId com.oluchi.panel9,
extraResources profiles/ + helper/Panel9 Direct.exe)
build/icon.png|ico      app icons
profiles/*.json         bundled DEVICE profiles (iphone-15-pro, pixel-8, galaxy-s24-*, ipad-pro-11,
iphone-se)
src/main.js             main process (~1100 lines): windows, tabs, layout, menus, shortcuts, sessions,
downloads, IPC, update flow, startup
src/config.js           store connection: STORE_API, APP_ID 'panel-9', LEGACY_APP_ID 'mobile-browser',
PLATFORM, startup delay
src/store.js            data layer: ProfileManager + ProfileData (settings, containers, history,
bookmarks, downloads) as JSON files
src/profiles.js         loads/validates device-profile JSON from several folders, fills defaults,
{chromeMajor} templating
src/accounts.js         best-effort detection of the Google account(s) signed in inside a container
(cookies + ListAccounts)
src/identity.js         ONE desktop-Chrome identity (UA, Sec-CH-UA brands, platform) shared by headers,
JS and tests (pure Node)
src/identity-preload.js frame preload registered on every session: makes window.chrome /
navigator.userAgentData look like real Chrome (Google sign-in)
src/extensions.js       ExtensionManager: registry, store install/update, load into every session,
storage sync (pure Node, Electron injected)
src/zip.js              dependency-free zip reader/extractor (zip-slip, symlink and size guards) +
builder for tests
src/ctxmenu.js          PURE context-menu template builders
(page/link/image/media/editable/tab/strip/bar/toolbar/extension/internal rows) — unit-tested; main.js
supplies state + actions (§13)
src/proxies.js          PURE proxy logic: line/bulk parsing, ProxyStore (safeStorage-injected
encryption), Chromium proxy rules, country→tz/locale table, geo answer parser (§12)
src/socksbridge.js      loopback SOCKS5 bridge that logs in to an authenticated upstream SOCKS5 proxy
(Chromium can't send SOCKS5 credentials) (§12)
src/emulation.js        per-tab CDP (webContents.debugger) emulation: metrics, touch, UA + client hints,
hw concurrency
src/inject.js           script injected before page scripts in every frame (navigator.*, screen.*,
```

```
matchMedia, WebGL strings, Safari clean-ups)
src/directhelper.js     DirectHelper: installs + supervises the "Panel9 Direct" exe (copy to
userData/helper/, spawn with P9D_TOKEN env, ready line, restart back-off, stdin-close shutdown) (2.3.6)
src/expressvpn.js       ExpressVpn: wraps ExpressVPN's own CLI `expressvpnctl`
(status/connect/disconnect/regions/split-app) + tolerant parsers; never fakes a result (2.3.6)
helper/p9direct/        Go source (no deps) of the helper: loopback HTTP+SOCKS5 proxy with token auth.
tools/build-helper.js -> dist/helper/Panel9 Direct.exe (+ p9direct-linux for tests)
src/lounge.js           Lounge storage (pure Node): per-profile file space, path validation, chunked
upload sessions (2.3.6)
src/loungeglue.js       Lounge glue: stash, editor tabs, live-preview server, export/import, api9
lounge.* / editor.* (2.3.6)
src/viewguard.js        view guard helpers: blank-content check, uniform-frame check, compositor repaint,
error texts (2.3.6)
src/webapp/state.*      matte loading skeleton / error card (with Retry) drawn over a web-app view
(2.3.6)
src/vendor/             offline bundles built by tools/build-vendor.js: CodeMirror 6, Prettier, pdf.js,
pdf-lib (no CDN) (2.3.6)
src/launcher.js         Starts updater helpers OUTSIDE the app's process job (WMI -> scheduled task ->
spawn) and waits for a handshake file (2.3.7)
src/updater.js          App Store self-update (check, download+SHA-256 verify, Windows .bat swap script,
legacy rename)
src/migrate.js          one-time import of old "Mobile Browser" userData into the first Panel 9 profile
src/error.html          in-tab load-failure page
src/ui/                 browser chrome (tab strip, toolbar, address bar): index.html, ui.js, ui.css,
preload.js (whitelisted `browserAPI`)
src/pages/              internal pages served via panel9:// : extensions, newtab, history, downloads,
bookmarks, bookmarkedit, accounts, proxies (Location & proxies), settings, picker (profile chooser) +
p9.js/p9.css shared helpers, preload.js (`window.P9`)
test/                   verify.js (538 checks; includes ctx-checks, accounts-checks, proxy-checks),
updater-checks.js, p9-checks.js, ext-checks.js (extension platform), ctx-checks.js (context menus, real
right-clicks), accounts-checks.js (>10 Google accounts, fake Google), proxy-checks.js + proxy-servers.js
(local HTTP/SOCKS5 test proxies), standalone.js <module> (run one module), shots-ctx.js (real-menu
screenshots), google-live.js (network: Google sign-in page loads in 4 paths), yt-focus-*.js (network:
YouTube Focus on real youtube.com), update-live.js (hits the live store), shots.js (regenerates ../shots)
shots/                  store screenshots (published as store screenshots)     README.md  user/dev docs
(long; emulation details, device-profile format)
```

( `dist/` , `release/` , `node_modules/` are build outputs/deps and are not in the source zip.)

## 4. Key modules and data flow

- **Window model ( `main.js` )** — one `ctx` per window: `{win, profileId, data (ProfileData), tabs[], activeTab, isPrivate, panel…}` . The chrome UI is one `WebContentsView` ( `src/ui` ), every tab is its own `WebContentsView` (sandboxed, `contextIsolation` , no Node, **no preload for web content**). `layout()` places toolbar/bookmarks bar/tab view; `phoneLayout()` draws the phone frame in mobile mode.
- **UI ⇄ main IPC** — `src/ui/preload.js` exposes `browserAPI.invoke(action,arg)` limited to an `ACTIONS` whitelist → `ipcMain.handle('ui:<name>')` → `handlers[...]` ; main pushes `state` snapshots ( `pushState` ) to the UI. Internal pages use `window.P9.call(method,args)` → `ipcMain.handle('p9:call')` → `api9[method]` , and the main process re-validates the sender URL (must be `panel9://` ).
- **`panel9://` protocol** — registered as privileged scheme; `handlePanel9()` serves only whitelisted page names/extensions from `src/pages` with a strict CSP ( `script-src 'self'` , no inline script, `connect-src 'none'` ). New internal page = add `<name>.html` + `<name>.js` , add name to `PAGE_NAMES` , add API methods in `api9` .
- **Profiles & containers ( `store.js` )** — `userData/panel9.json` (list, last profile, launch mode) + `userData/users/<id>/{settings,containers,history,bookmarks,downloads}.json` ( `JsonFile` = debounced atomic writes). A **container** is a named/coloured isolated session: Electron partition `persist:p<profileId>-c<containerId>` ( `partitionName()` ); the first container ( `default` ) is the profile's main session. Private windows use in-

memory partitions wiped on close ( `wipeSessions` ). Deleting a profile/container queues `pendingDelete` and the partition data is purged at next start ( `purgeDeletedData` ).

- **Accounts ( `accounts.js` )** — detects "signed in" via Google auth cookies and e-mail via `accounts.google.com/ListAccounts` using the container's own session; shown in the Accounts panel. Nothing leaves the container.
- **Tabs** — `createTab(ctx,url,mode,{container})` ; popups/OAuth windows are attached to the opener's session ( `attachPopup` ) so Google sign-in works; `setTabMode()` toggles desktop/mobile per tab.
- **Emulation** — mobile tab = `emulation.js` CDP overrides + `main.js` `session.webRequest.onBeforeSendHeaders` rewriting `User-Agent` , `Accept-Language` , `Sec-CH-UA*` + `inject.js` . Desktop tab = a normal Chrome UA (no "Electron") and everything cleared. Device profiles: JSON, `{chromeMajor}` templated, loaded from bundled `profiles/` , next to the portable exe, and `%APPDATA%\Panel 9\profiles` (later wins by `id` ).
- **Downloads** — `onWillDownload` per session, saved to Downloads (or ask), tracked in `downloads.json` , shown in the toolbar/Downloads page.
- **Updater ( `updater.js` + `checkForUpdates()` in main.js)** — see §7.
- **Migration ( `migrate.js` )** — first run copies settings, custom device profiles, and `Partitions/browser` cookies from `%APPDATA%/Mobile Browser` into profile `1` ; guarded by `migrated-from-mobile-browser.json` .
- Startup ( `ready` async block): migrate → `ProfileManager` → purge deletions → load device profiles → picker (if several profiles and "ask on startup") or open last profile → `afterStart()` (update check after 4 s, legacy-rename offer after 9 s).

## 5. Build and run

```
cd /workspace/mobile-browser
npm install
npm start                                  # dev run
npm run test:verify                        # xvfb-run electron test/verify.js (538 checks) — run before
every release
node test/updater-checks.js                # pure-node updater tests
npm run test:google                        # network: Google sign-in loads (desktop, container,
private, mobile)
npm run test:ytf                           # network: YouTube Focus extension against real youtube.com
(needs ../yt-focus-extension)
npm run shots                              # regenerate ./shots (needs xvfb, ffmpeg)
npm run build:helper                       # go build -> dist/helper/Panel9 Direct.exe (+ p9direct-
linux); `dist` runs it first
npm run test:helper|test:vpn|test:ctlvpn|test:tabmove|test:ctltabs   # 2.3.6 suites (part of
test:modules)
npm run dist                               # electron-builder --win portable nsis --x64 ->
release/Panel 9-<v>-portable.exe and -setup.exe
```

Env switches: `P9_NO_UPDATE_CHECK=1` , `P9_NO_EXT_UPDATE=1` , `P9_STORE_API=<url>` , `P9_PROFILE=<id>` , `P9_LEGACY_DIR` , `P9_ACCOUNTS_URL` (tests). Exe is unsigned (SmartScreen warning).

## 6. Publishing to the App Store

Bump `version` in `package.json` → `npm run dist` → publish (through `bash -lc` ; the backend runs on the DigitalOcean droplet, `publish.sh` uploads the exe to R2 then pushes metadata there):

```
bash -lc '/workspace/appstore/bin/publish.sh panel-9 <version> "/workspace/mobile-browser/release/Panel
9-<version>-portable.exe" "Release notes" \
  --platform windows --source /workspace/mobile-browser --arch /workspace/mobile-
browser/ARCHITECTURE.md'
```

**Also publish the same exe under the legacy id** so Mobile Browser ≤1.1.x installs (which poll `mobile-browser` ) upgrade: `publish.sh mobile-browser <version> "<same exe>" "<notes>"` (manifest `mobile-browser` is `hidden:true` , `superseded_by:panel-9` , so it is not listed in the store UI but `/latest` + `/download` keep working). Metadata-only refresh of

source/architecture for an existing version (no new binary): `publish.sh panel-9 2.1.0 - --source /workspace/mobile-browser --arch /workspace/mobile-browser/ARCHITECTURE.md`.

## 7. Update-check contract

`GET {STORE_API}/api/apps/panel-9/latest?platform=windows&current=<version>` (`STORE_API` from `src/config.js`, default `https://api10.axionswap.com`; was api10.norhartpro.com until 2.3.27). Used fields: `version`, `url`, `sha256`, `size`, `filename`, `notes`, (`update_available`). Flow: `updater.checkForUpdate` (own semver compare) → dialog with notes (Update now / Later / Skip this version) → `downloadAndVerify` follows the **302 to a presigned R2 URL**, streams to `<dir>/<name>.download`, verifies size + **SHA-256**, deletes on mismatch → `applyUpdate`: portable exe ⇒ spawn a temp `.bat` that waits for the process (and electron-builder's portable launcher, via `PORTABLE_EXECUTABLE_FILE/DIR`) to exit, swaps/renames the exe and relaunches; installed (NSIS) ⇒ opens the downloaded installer. A failed download never touches the current exe. `skippedVersion` is stored per profile settings. Legacy installs of "Mobile Browser" call `/api/apps/mobile-browser/latest` and get the Panel 9 exe (`filename` `Panel 9-<v>-portable.exe`); `legacyRenameTarget()` + `offerLegacyRename()` then offers to rename an old-named exe once. The store API also exposes `/api/apps/panel-9/source` (this project as a zip) and `/architecture` (this file).

## 8. Gotchas

- **Never** load `panel9://` pages or the preload bridge into web content; keep the sender-URL check in `p9:call`.
- CSP for internal pages forbids inline scripts/styles — put JS/CSS in files.
- Partition names are persistent identifiers (`persist:p<id>-c<id>`); renaming breaks users' logins.
- `package.json` `version` drives both the update comparison and the exe name; the store manifest `file` must match the built file; `sha256` is computed by `publish.sh` from the exact file.
- The `mobile-browser` store id must keep receiving each new Panel 9 exe (hidden) or legacy installs stop updating; conversely do not un-hide it or the store lists the app twice.
- Google sign-in from embedded browsers can still be challenged (risk-based, per IP/account); popups must remain in-app child windows sharing the container session. Do not add a second source of truth for the browser identity — everything goes through `src/identity.js`.
- Electron can only load extensions into **persistent** sessions: private windows opened while an extension is enabled use a disposable `persist:p9priv…` partition that is wiped on close (§10). Extension files must not start with `_` (Chromium reserves those names), which is why the host page is `p9host.html`.
- Windows-only behaviours (`.bat` swap, portable env vars) can't be exercised on the Linux box; `test/update-live.js` and `updater-checks.js` cover the logic, the rest needs a Windows check.
- No secrets in the repo: the store admin token and R2 keys are never required to build; don't commit any.

## 9. Google sign-in ("This browser or app may not be secure")

**Cause.** accounts.google.com fingerprints the page, not just the UA. Panel 9 sent a genuine Chrome UA but Electron's renderer has an incomplete `window.chrome` (no `loadTimes` / `csi` / `app`) and its `navigator.userAgentData` brands did not match the `Sec-CH-UA` header, so Google classified it as an embedded app and refused. (Reproduced on the Linux box with a dummy e-mail; bisected: UA alone is not enough, the missing `window.chrome` members plus the brand mismatch trigger the block.) **Fix.** `src/identity.js` is the single identity (Chrome major from the running Chromium, brands `Not:A-Brand/Chromium/Google Chrome`, Windows platform). `src/identity-preload.js` is registered with `ses.registerPreloadScript({type:'frame'})` on every session (profiles, containers, private, popups): it only runs when the UA is a desktop Chrome UA (never in mobile emulation) and adds `chrome.loadTimes/csi/app` and a consistent `NavigatorUAData` (`brands`, `getHighEntropyValues`, `toJSON`) with native-looking `toString`. `emulation.js` always rewrites `Sec-CH-UA`, `-Mobile`, `-Platform` from the same source. For `accounts.google.com`, **mobile emulation is suspended for the tab** (`suspendEmulationForGoogle`, resumed when the tab leaves Google auth) because Google also rejects iPhone-Safari UAs with desktop engines; the account page is therefore always seen as desktop Chrome, in normal profiles, containers and private windows. **Verification.** `npm run test:google` loads `accounts.google.com` sign-in with a dummy address in a normal profile, a container, a private window and a mobile-mode tab and asserts the e-mail form is shown and no "not secure" text appears (screenshots in `shots/google-signin-*.png`). It cannot prove the outcome for a real account: Google's risk engine is also IP/account dependent and the test runs from a datacentre IP; Windows was not available on the box (the box reports a Linux `navigator.platform`).

## 10. Extensions

- **Package format.** Unpacked Chrome extensions (MV2/MV3): `manifest.json` at the zip root, content scripts (incl. `"world":"MAIN"`), `chrome.storage`, action popup / options page. Electron's `session.extensions` implements only a subset of Chrome APIs (no `chrome.tabs` UI, no web store), so keep extensions to content scripts + storage + popup.
- **Store.** Extensions are normal store apps with `type: "extension"` and `target: "panel-9"` (backend `/api/apps?type=extension&target=panel-9`, `/api/extensions`); the Panel 9 binary has `type: "app"` (default for older entries). Publish with `publish.sh <id> <ver> <zip> "notes" --type extension --target panel-9 --platform any --source … --arch …`.
- **`ExtensionManager` (`src/extensions.js`).** Registry `userData/extensions/registry.json`, files in `userData/extensions/<id>/app` (id `local-<slug>` for "Load unpacked", which *copies* the folder). Install = `GET latest` → `updater.downloadAndVerify` (size + SHA-256) → `zip.extractZip` into `staging` → manifest version must equal the store version → atomic swap. Auto-update: 6 s after start and every 6 h (`startAutoUpdate`), toggled on the page; failed/corrupt downloads leave the installed version untouched. All operations run through a serial queue.
- **Every session.** `setupSession()` (called for each profile partition, each container partition — including containers created later — and private windows) calls `extMgr.attachSession(ses)` and `loadExtension()`s every enabled extension; tab creation awaits `sesExtReady` so content scripts exist before the first navigation. Electron cannot load extensions into in-memory sessions, so while at least one extension is enabled a new private window uses a disposable `persist:p9priv<tag>` partition (deleted on close and purged at start); a private window opened *before* the first extension was enabled keeps working without extensions and shows a notice on the toolbar button.
- **Shared `chrome.storage`.** Each session has its own extension storage, so a hidden extension page (`p9host.html/js`, injected into every copy) reports `chrome.storage.local` changes through a tagged `console.log`; the main process mirrors them into a per-extension JSON (`_storage/<id>.json`) and writes them into all other sessions. Net effect: a toggle flipped in one profile/container applies live to all open tabs everywhere and is seeded into new sessions.
- **UI.** 🧩 toolbar button (menu: extensions list, "Manage extensions", "Load unpacked…") + one button per enabled extension with a popup. A popup opens in a small `BaseWindow` (380×560, auto-sized to content, Esc closes) using the active tab's session; the page `panel9://extensions` lists Installed / Store with Install, Remove, Enable/Disable, Reload (unpacked), "Check for updates" and an auto-update switch. After enabling/updating, tabs whose URL matches the content-script patterns are reloaded (`reloadMatchingTabs`).
- **Tests.** `test/ext-checks.js` (in `verify.js`): zip + match patterns, fake store (install, sha256 failure, update on/off, wrong target/type), loading into profile/container/new container/private window, popup toggle → live tab update in all sessions, disable/enable/remove, registry persistence, load unpacked + reload.

## 11. More than 10 Google / YouTube accounts in one profile

**Problem.** Google's multi-login ("Use another account") holds at most **10 accounts per cookie jar**. A profile's Main session and each account container is one cookie jar. **Design (all built into Panel 9, no extension needed).**

- **Automatic overflow** (`main.js` `maybeOverflowAddAccount` on `will-navigate` / `will-redirect`, `lateOverflowCheck` on `did-start-navigation`, `overflowSignIn`): when a tab whose container already shows ≥ `accounts.cap()` (10; env `P9_GOOGLE_CAP` in tests) accounts goes to an "add account" URL (`accounts.isAddAccountUrl`: `/AddSession`, `flowEntry=AddSession`), the navigation is cancelled and the same sign-in opens in a container of the same profile named `Google 2`, `Google 3` … (`accounts.overflowTarget` reuses one with room, otherwise creates the next). A notice explains it. The user never has to think about containers.
- **One unified list** (`accounts.allPeople(ctx)`): every container's `ListAccounts` people (`parsePeople`: name, email, avatar, `index` = Google `authuser`) are cached in container meta (`store.accountMeta.people`, ≤50 per container, https avatars only) and merged.
- **One switcher.** Toolbar 👥 panel → "Switch Google account" card (search, avatar/name/email/container badge, **Switch**; "Add account" button = `accounts.add`), menu item *Switch Google account…* (Ctrl+Shift+K) and an injected row **"All accounts (Panel 9)…"** in YouTube's/Google's own account menu (`pages/preload.js`, top frames of youtube.com/google.com only, trusted clicks only). Switching = `switchTabToAccount`: the current tab moves to the container holding the account (same URL via `?authuser=N` where needed, same slot, pin state kept).
- **Bridge for extensions/pages.** `window.postMessage({type:'panel9:open-account-switcher'})` from a youtube/google page opens the switcher **only while** `navigator.userActivation.isActive` (real user gesture); the preload sets `data-panel9-switcher="1"` on `<html>` so extensions can feature-detect. The page never receives account

data back (one-way). `ipcMain.on('p9:switcher')` re-validates sender tab, main frame and host. yt-focus's comments-only *Switch account* can call this to reach accounts beyond 10.

- Test hooks (TEST only): `P9_GOOGLE_HOSTS`, `P9_GOOGLE_BASE`, `P9_ACCOUNTS_URL`, `P9_BRIDGE_HOSTS`. `test/accounts-checks.js` runs a fake Google (`/ListAccounts` from cookies) with 14–24 accounts. **Untested against real Google:** the real cap error text, the exact ListAccounts shape/YouTube menu markup (selectors are defensive), brand-channel switching (YouTube's own list per account still applies).

## 12. Proxies / locations (VPN-like, per profile)

**Why built in.** Chrome-style extensions cannot do this in Electron: there is no `chrome.proxy` API (nor `webRequest` proxy auth hooks), so per-profile proxies live in the app. It is independent of any VPN app running on the PC (ExpressVPN etc. keep working for "Direct"). **Data.** `userData/proxies.json` (global list): `{id,label,type http|https|socks5,host,port,username,pw (safeStorage ciphertext),country,tz,lastTest}`. When no OS key store exists (e.g. Linux `basic_text`) the password is kept **in memory only** (never written; user re-enters after restart; the page says so). Assignments live in profile settings (`proxyId` '' = Direct, `proxyKill` default **true**, `proxyAlign` default true, `privateProxy` inherit|direct|id) and container items (`proxy` ''=inherit|'direct'|id). **Resolution** (`main.js resolveProxy(ctx,cid)`): container override > private-window setting (private windows only) > profile proxy > Direct. Private windows read the *base profile's* settings so changes apply live. **Applying.** Every session is configured when created (`Ctx.sessionFor` → `initSessionProxy`; tabs wait for `sesProxyReady` before the first request, so there is no unproxied first load) and re-applied on any change (`reapplyProxies`: profile, containers, per-window chrome sessions, popups; old connections are closed). Direct = `{mode:'system'}` (unchanged legacy behaviour, respects Windows proxy/VPN). Proxied = `fixed_servers` with `http://|https://|socks5://host:port` — `socks5://` means **the proxy resolves host names (no local DNS)**; HTTP proxies get the host name in CONNECT/absolute-URI. **Kill switch** = rules **without** `,direct://` (dead proxy ⇒ `ERR_PROXY_CONNECTION_FAILED`, never the PC's IP); off = `…,direct://` fallback. A deleted-but-referenced proxy fails closed (dead-end SOCKS rule) and the button shows "!". The window chrome (toolbar favicons) and overlay panels use per-window sessions (`chromeSession`) so they take the profile's route too. **Proxy login.** HTTP/HTTPS proxies: `app.on('login')` with `authInfo.isProxy` answers from the store with no prompt (entry chosen by tab → container → proxy id when several share host:port; the same request asking again 3× = rejected → give up, notice + red toolbar button; wrong passwords never loop). **SOCKS5 with a login**: Chromium cannot send SOCKS5 user/pass, so `socksbridge.js` runs a no-auth SOCKS5 server on `127.0.0.1:<random>` per proxy that authenticates upstream (host names are passed through unresolved). Credentials are only in memory; nothing logs them. **Leak protection.** `webContents.setWebRTCIPHandlingPolicy('disable_non_proxied_udp')` on every proxied tab/popup/extension popup (HTTP proxies carry no UDP ⇒ no WebRTC UDP at all — video calls/STUN-based apps may not work through such a profile; that is the price of no leak). Time zone/language alignment (toggle): CDP `Emulation.setTimezoneOverride` + `setLocaleOverride`, a new-document script for `navigator.language(s)` (needs `Page.enable`), and session Accept-Language via `ses.setUserAgent(ua, langs)`; skipped on Google sign-in pages (which stay plain Chrome). Country→tz/locale table in `proxies.js`; the Test result's own time zone wins. **UI.** `panel9://proxies` (Menu → *Location & proxies…*): effective location of this tab, assignment card (profile / private windows / kill switch / alignment / per-container selects), proxy list (Test, Use for this profile, edit, delete, Test all, "Check this PC's own connection"), add form (+ "Test before adding"), **bulk paste** (`host:port`, `host:port:user:pass`, `user:pass@host:port`, optional `http://|https://|socks5://` prefix; bad lines stay in the box, passwords are never echoed). Toolbar **location button** (country code, green when proxied, red when failed/missing, 🌐 when Direct) follows the active tab's container; click (or Ctrl+Shift+L) = quick switcher (Direct + every proxy, per-account override submenu, kill switch, alignment, Test, Manage…). **Test** (`testProxy`) uses a throw-away session through the proxy (never falls back to direct) against `https://ipwho.is/` (fallback `ipapi.co`), reporting exit IP, country, latency; the result also fills the country. **Known limits.** The extension store/update checks and the profile picker window use the PC's connection (they are not browsing). `session.fetch` /downloads/popups of a tab's session are proxied. Windows behaviour of safeStorage (DPAPI) and real proxy providers are untested (box has none). **Tests.** `test/proxy-checks.js` (+ `proxy-servers.js`): real local HTTP(Basic auth, CONNECT) and SOCKS5 (user/pass and none) proxies dialing out from 127.0.0.11/.12/.13, an http+https target reporting the source address, a UDP STUN counter. Verifies parsing/bulk, encrypted/in-memory password storage, Test results, four simultaneous exit routes (2 profiles + 2 containers, one Direct), silent login, https CONNECT, DNS-at-proxy (`ghost.test`), later-created containers, mobile tabs, popups, private overrides, WebRTC baseline-vs-proxied, kill switch on/off/missing/wrong password, tz/language, button/quick switcher, the page and its form/bulk/Test, no secrets in API/disk/logs. Test env: `P9_PROXY_LOOPBACK=1` (route 127.0.0.1 through the proxy), `P9_IPGEO_URL`.

## 13. Context menus (right-click everywhere)

**Rule.** Every surface gets a Chrome-like native menu; nothing relies on the default (which on Electron is *no menu at all*). Templates are pure functions in `src/ctxmenu.js` (`buildPageMenu`, `buildEditMenu`, `buildTabMenu`, `buildStripMenu`,

`buildBarItemMenu`, `buildBarMenu`, `buildToolbarMenu`, `buildExtButtonMenu`, `buildHistoryMenu`, `buildInternalMenu`); `main.js` (context-menu block) supplies the state/actions and pops them through `popupMenu()` (in TEST mode menus are captured in `api.menus.captured` instead of shown unless `P9_TEST_MENU_POPUP=1`).

- **Web pages** (`wc.on('context-menu')` in `createTab`, `attachPopup`, extension popups): link (open in new tab/window/private window/another account, save link as, copy address), image (open, save, copy image, copy address), selection (copy, Search Google), editable (undo/redo/cut/copy/paste/paste as plain text [rich fields]/select all + spellcheck suggestions/add to dictionary), media (play/pause, mute, loop, controls, save), page (back/forward/reload, save as, print, add to favourites, switch mobile/desktop, this frame, view source), always *Inspect*. Pages with their own `contextmenu` handler still get ours unless they `preventDefault` natively (Electron then doesn't fire the event — same as Chrome).
- **Mobile-emulated tabs**: touch emulation delivers the right-click without a hit target, so `mobileHitTest` re-hit-tests the page via CDP before building the menu.
- **Tab strip** (`ui.js` global `contextmenu` listener → `ui:tabCtx/stripCtx/…`): new tab to the right, reload, duplicate, pin/unpin, mute, switch mode, close/close others/close to the right, reopen closed tab (Ctrl+Shift+T), move to new window; strip: new tab, reopen, bookmark all tabs. **Address bar** (async clipboard in Electron 45): undo/cut/copy/paste/paste and go/select all. **Favourites bar** items/bar, **toolbar**, **extension buttons** (popup/options/disable/remove), **back/forward** (history list).
- **Internal pages** mark rows with `data-ctx` JSON (`p9.ctxAttr`); `p9.js` forwards the right-click via `P9.call('ctx.show')` → `showInternalMenu` (history, tiles, favourites, downloads, extensions, store, profiles, containers). Page-side actions (rename, edit) come back as `p9:event {type:'ctx'}` → `p9.onCtx`.
- Menus keep overlay panels open (`ctx.menuOpen`) so the panel's blur handler doesn't close them. Shortcuts added: Ctrl+Shift+T, Ctrl+U, F12/Ctrl+Shift+I, Ctrl+Shift+K, Ctrl+Shift+L.
- **Limits:** favourites-bar *folder children* are a native submenu without their own menu; `chrome.contextMenus` from extensions is not supported; "move to new window" re-opens by URL/mode/container (a private tab loses its session state); native popups can't be captured by `capturePage` (screenshots use x11grab: `test/shots-ctx.js`); Windows' native menu look is untested.
- **Tests.** `test/ctx-checks.js` (107): template unit checks + real `sendInputEvent` right-clicks in every session type (profile, container, mobile, private, second profile), popups, panels, internal pages.

## 2.3.9: Bean bridge (`window.panel9ext.containers()` / `openTabInContainer()`)

The **Bean** extension (store id `panel9-bean`, `/workspace/panel9-bean`) likes pasted YouTube comment links from several account containers. Electron's extension subset has no `chrome.tabs`, so 2.3.9 adds two methods to the existing privileged bridge (`src/extbridge.js` + `src/ext-bridge-preload.js`, IPC `p9ext:containers` / `p9ext:openTab`), wired in `src/main.js` (`extbridge.install({... listContainers, openTabInContainer})`):

- **Allow-list**: only extension ids matching `BEAN_ID_RE = /(^|-)panel9-bean$/` (store install `panel9-bean`, unpacked `local-panel9-bean`) and only the **popup's main frame** (`chrome-extension://<id>/`, same rule as the Authenticator/Notes/PIA entries). Other extensions get "not allowed" for both methods.
- `containers()` → `{profile:{id,name,isPrivate}, active, containers:[{id,name,color,builtin,signedIn,checked,people:[≤3 names]}]}`. `active` = container of the window's active tab. No e-mail addresses, cookies or partition names.
- `openTabInContainer({containerId, url})` → `createTab(ctx, url, 'desktop', {container})` in the same profile window; returns `{tabId, containerId}`. `url` must match `YT_WATCH_RE = ^https://www.youtube.com/watch?[^\s#]{1,380}$`; unknown/missing container ⇒ error and **no tab**; rate limit 30 opens per 60 s per window (`openLog`).
- Channel discovery (Bean's "Refresh channels"): `YT_WATCH_RE` also accepts `https://www.youtube.com/?<query>` (the home page, so the tab can read YouTube's own account switcher), and `closeTab({tabId})` closes **only tabs this bridge opened itself** (`openedTabs`, per window, max 500; user tabs and other extensions are refused).
- **Storage-mirror fixes (`src/extensions.js`)** that Bean's run record needs: `_push` sends only the delta against what the host last had (a whole replace only to seed a new host), echoes of its own pushes are ignored for 4 s (`pushedLog`), and `_onSnapshot` merges per key — before, a stale whole snapshot from another container's session could revert a newer value (a finished item went back to "running", a saved selection vanished).
- Test: `test/bean-bridge-checks.js` (`npm run test:bean`, part of `test:modules`; no network: www.youtube.com is redirected to a local server inside the test). Without this release the extension still works but only from the current tab's container.

## 2.3.0 — Exit IP chip (merged with the location button)

The toolbar always shows where the **active tab** appears to be: SVG flag, `City, CC` and the public IP (full mode) or flag + country code (compact; Settings → Appearance → Location chip, or the chip's menu). One control: click opens the location menu, whose top block is the exit info (place, IP — click to copy, ISP/ASN, time zone, Refresh, chip display) followed by the existing proxy choices, kill switch and align options.

- **Lookup** ( `src/exitip.js` + `refreshExit` in `main.js` ): `net.request` **on the tab's own session** (so it takes exactly that tab's route: profile proxy, container proxy, private-window proxy or Direct; proxy logins answered from the proxy store). Providers in order: `ipwho.is` → `ipapi.co` → `ip-api.com` (HTTPS first; the free ip-api plan is HTTP only, hence last). A dead proxy stops the provider loop (another provider will not help).
- **States:** `ok` , `loading` (last known value dimmed), `blocked` (kill switch on and the proxy is missing/unreachable/refuses the login: red chip, "Blocked", **no lookup is ever made around the proxy**), `offline` (name-resolution/disconnect errors or `net.isOnline()` false), `limited` (HTTP 429 / provider "rate limit" answer; last known value kept), `error` . With the kill switch OFF a dead proxy falls back to direct and the chip honestly shows the real IP.
- **Cache** per route key ( `direct | p:<proxyId>:<host>:<port>:<kill>` ), 5 min TTL, in memory only — switching tabs between routes costs no requests. **Refresh** on: tab activation (incl. container switches), any proxy/kill-switch/assignment change ( `reapplyProxies` clears the cache), UI `online` / `offline` events ( `netChange` ), every 5 minutes ( `P9_EXITIP_INTERVAL` in tests), manual "Refresh exit IP". A sequence number drops answers that arrive after the user already moved on.
- **Flags:** `src/theme/flags/*.svg` (country-flag-icons 3x2, MIT, licence alongside) — Windows has no flag emoji. Served to `panel9://` pages at `/flags/xx.svg` , to the toolbar via `file:` (CSP `img-src file:` ) and to menus as data URLs.
- **Tests:** `test/exitip-checks.js` ( `npm run test:exitip` ): real local HTTP/SOCKS5 proxies dialling out from different loopback addresses (the lookup server reports the source address), kill switch, deleted proxy, offline, 429, provider fallback, cache hit counting, compact mode, merged menu, private window. Screenshots: `test/exitip-shots.js` → `shots/exitip-*-{light,dark}.png` .

## 2.3.0 — Design system "Matte" and themes

- **Tokens:** `src/theme/tokens.css` is the single source of truth (colours, radii, motion, keyframes, scrollbars, Inter variable font from `src/theme/fonts` , reduced-motion rules). Matte (dark) is the default; Matte Light sits under `@media (prefers-color-scheme: light)` . The chrome UI, menus, dialogs, `panel9://` pages and built-in extensions all link it (extensions carry a copy as `theme.css` + `fonts/` ).
- **Scheme:** `nativeTheme.themeSource` ← app prefs `theme` ( `dark` default | `light` | `system` ) or the active store theme's `scheme` . That alone drives `prefers-color-scheme` everywhere; no JS needed in pages.
- **Store themes** ( `src/themes.js` , `ThemeManager` ): store items `type: "theme"` (zip = `theme.json` + `preview.png` , `target(s)` panel-9[,yt-panel]); installed to `userData/themes/<id>/` , sha256 verified via `updater.downloadAndVerify` , auto-updated every 6 h like extensions. Tokens are whitelisted/validated ( `ALLOWED` , `validToken` : no `url()` , `@import` , braces, semicolons). `cssFor({theme, accent, motion})` builds one stylesheet that `main.js` ( `applyThemeCss` / `applyThemeEverywhere` ) inserts with `insertCSS` into Panel 9's **own** web contents only ( `ownWc` : `panel9:` , `file://` under `src/` , `chrome-extension://` pages of panel9 extensions) — never into websites. Accent ( `themes.accent` ) and `motion=off` ride on the same stylesheet.
- **Picker:** `panel9://themes` ( `src/pages/themes.*` ): live-preview cards for Matte / Matte Light / System, installed themes, accent swatches, animations switch, "From the App Store" grid (Install / Update / Apply). Handlers `themes.get/apply/accent/motion/store/install/remove` . Settings → Appearance mirrors the basics.
- **Shared spec:** `/workspace/appstore/THEMES-SPEC.md` + `templates/theme-client.js` (app-agnostic; YT Panel uses the same format). Panel 9 accepts a theme when `targets` (or `[target]` ) contains `panel-9` .
- **Tests:** `test/theme-checks.js` ( `npm run test:theme` ): validation units, Matte/Matte Light flip, a mock store (zips + sha256), install, apply live in page + chrome, light store theme, accent, motion off, settings controls, removal fallback, error page. `test/design-shots.js` and `test/theme-previews.js` produce the screenshots ( `shots/design-*.png` , store theme previews). NB screenshot with a *visible* window (xvfb + x11grab): hidden windows do not run CSS animations.
- **Error / offline / crash pages:** `src/error.html` (kind-specific: offline, DNS, proxy/kill-switch, certificate, timeout, refused, crash) uses the same tokens; renderer crashes load it with `kind=crash` .
- **Not done in this batch (honest list):** frameless custom Windows title bar, NSIS installer branding, a custom replacement for every native `<select>` , and a full per-surface animation audit — see the release report.

## 2.3.0 — PIA VPN (extension `panel9-pia` + `src/pia.js`)

Electron has no `chrome.proxy`, so PIA is a *manager of Panel 9 proxy entries*, never a second routing layer.

- **Public flow only (same as PIA's MIT-licensed extension):** `POST https://www.privateinternetaccess.com/api/client/v2/token {username,password}` → `{token}` (24 h); `GET https://serverlist.piaservers.net/proxy` → `[{name, iso, dns, port, mace, ping}]` (no login); HTTPS proxy `<dns>:443` with `Proxy-Authorization` = token split in half (first half user, second half password). A bundled snapshot `src/pia-regions.json` is the offline fallback.
- **`src/pia.js`** (pure Node): `login/refresh/logout`, region cache (60 min) + latency (TCP connect to the region's proxy port, 8 parallel), favourites, `ensureEntry(region)` → one managed `ProxyStore` entry per region actually used (`managed:'pia'`, `region`). `ProxyStore` hides the login half (stored encrypted as `un`, memory-only without an OS key store), pages cannot edit managed entries (`proxies.update` refuses), and `addManaged` is only reachable in-process (Symbol-guarded).
- **Credentials:** username/password typed in the extension popup go to the main process over the `p9ext:pia` bridge (authorised only for ids matching `(^|-)panel9-pia$`), are sent to PIA only, and the password is kept only when "Remember me" is ticked (safeStorage; memory-only if no key store). Token/password are never logged (asserted by the test).
- **Assignment** reuses the proxy manager: `piaCall` ops `assign` (scope `profile|container|private`) → `setProfileProxy` / `setContainerProxy` / `privateProxy`. So the **kill switch, WebRTC policy, time zone + language follow, per-container independence and the exit chip** are the existing machinery (the chip's geo lookup runs through the PIA route).
- **Token refresh:** at 20 h (`needsRefresh`, 10-min timer + start-up), and on a proxy login rejection (`onProxyLogin` → `piaAuthRejected`, throttled 5 min). Without a remembered password an expired session says "sign in again" instead of looping.
- **Location menu:** "PIA VPN regions…" opens the popup (or "Get the PIA VPN extension…").
- **Tests:** `test/pia-checks.js` (`npm run test:pia`) against a **mock** of auth + serverlist + three token-authenticated HTTP proxies (shared mutable token halves; `pia.ep` / `pia.proxyType` overridden by the test). **Unverified (no PIA account exists):** real sign-in, the real proxies accepting token-half auth, TLS to the real `https` proxies (the test uses `http` scheme), real latency/region data, PIA terms/rate limits.

## 2.3.0 — Notes extension (`panel9-notes`)

MV3 extension, popup + full-page tab (`app.html?view=popup|full`). Data in `chrome.storage.local` (mirrored to every session by Panel 9; per-profile keys `p9notes.v.p.<profileId>` or shared); rich text/markdown, checklists, images (downscaled), notebooks/tags/pin/fav/trash, search, site-linked notes (bridge `activeTab`), clip inbox via content script (Alt+Shift+N), exports (Markdown zip, JSON, optional password), PIN lock sealing all datasets. The bridge (`src/extbridge.js`) now supports several bridged extensions with per-extension key caches and tab-hosted extension pages. Tests: `test/notes-checks.js` (57 checks).

## 2.3.0 — Screenshots / screen recording

`src/capture.js` + `src/capture/*` + `src/pages/editor.*` + `captures.*`: region/visible/full-page screenshots with an annotating editor, screen & window recording with a countdown/pause banner. Tests: `test/capture-checks.js`.

## 2.3.6 — Tab repositioning (drag, keyboard, menu, p9ctl) and session restore

- **Strip drag** (`src/ui/ui.js`, pointer events + pointer capture on `#tabs`): after 5 px the tab lifts into an in-strip ghost; the other tabs glide (FLIP) and a slim `#dropbar` shows the slot. Gap indexes are computed WITHOUT the dragged tab, so the gap number is the final index (`to = gap`). Pinned tabs form a group in front: a pinned tab can only land among pinned slots, an unpinned one only after them (clamped in `reorderTab`, main side, so UI/keyboard/p9ctl agree). Esc cancels.
- **Tear-off**: when the pointer leaves the strip vertically (>26 px, `TEAR_PX`) the UI sends `tabDrag` phases (`start|move|end|cancel`) to main (`tabDragEvent`). Main shows a small frameless always-on-top ghost window (`src/ui/ghost.html`) that follows the cursor and hit-tests other Panel 9 windows of the SAME profile (their strip lights up). Drop on another strip -> `adoptTab`; drop elsewhere -> `detachTab` = new window at the cursor. Both **move the live WebContentsView** (`t.rebind(newCtx)`): no reload, JS state/scroll/media survive, the container partition (`persist:p<profile>-c<container>`) and profile stay. Private tabs never detach / are never targets; popups of a moved tab stay with the old window; moving the last tab closes the emptied window.

- **Keyboard/menu**: Ctrl+Shift+PageUp/PageDown = `moveTabBy(-1/+1)` (handled in main's before-input-event so it works while the page has focus). Tab context menu ( `src/ctxmenu.js` ): Move tab left/right, Move to start/end, Move to new window, Move to window > (other windows of the profile). Items are disabled when they would do nothing ( `canMoveTab` ).
- **p9ctl**: `tab move <index|id> <newIndex|start|end|left|right> [--window W]` and `tab move-window <id> <windowId|new>` ( `src/control.js` `tab.move` , `tab.move-window` ). The `new` window is created INACTIVE and no command focuses/raises anything ( `test:ctlnofocus` rule of 2.3.5 is kept and asserted in `test:ctltabs` ).
- **Session restore** (new, default ON, per-profile setting `restoreSession` , Settings -> "Reopen tabs from last time"): `userData/session.json` holds, for every normal window, its tabs IN STRIP ORDER (url, mode, container, pinned, back/forward history), the active tab and bounds. Saved (1.2 s debounce) on tab changes and at quit; restored on first window of a profile. Reordering therefore survives a restart. Private windows are never stored. Tests enable it with `P9_TEST_SESSION=1` .
- Tests: `test:tabmove` (real pointer drag through `sendInputEvent` , tear-off, cross-window, restart in a child process), `test:ctltabs` (real relay + p9ctl), `ctx-checks` (menu).

## 2.3.6 — ExpressVPN + "Outside VPN" (extension `panel9-expressvpn` , `src/expressvpn.js` , `src/directhelper.js` , `helper/p9direct` )

Two independent parts. **Nothing is faked: whatever the real ExpressVPN app/CLI does not answer is reported as "unavailable" with the reason.**

1. **Control of the global ExpressVPN app** ( `ExpressVpn` , via `expressvpnctl[.exe]` , ExpressVPN for Windows >= 12.69, documented as "run as administrator"): `status` ( `get connectionstate|region|pubip` , `status` ), `connect` , `connect "<location>"` , `disconnect` , `get regions` (10-minute cache). Found via `P9_EXPRESSVPN_CLI` , saved path, `C:\Program Files (x86)\ExpressVPN\services\` , `C:\Program Files\ExpressVPN\` , PATH. `execFile` with an args array (no shell). Error classes `missing|admin|app|account|error` -> `state:'unavailable'` + `reason` + readable `message` and an "Open ExpressVPN" button. Parsers are tolerant (the exact Windows output format is UNVERIFIED on real Windows). A 20 s poll ( `P9_VPN_POLL_MS` ) feeds the toolbar chip ( `locState().vpn` ): connected location + flag; when the exit-IP chip is on the normal route it shows the VPN's location while connected.
2. **Outside VPN (split tunnel per window/container)**: ExpressVPN only splits per application, so Panel 9 ships a tiny helper, **`Panel9 Direct.exe`** (Go, `helper/p9direct` , ~5 MB, in `resources/helper/` of BOTH the portable and the setup build via `extraResources` ; `npm run dist` runs `build:helper` first). It is copied to the stable path `userData/helper/Panel9 Direct.exe` (portable builds run from a temp dir; the VPN rule is path based) and started by Panel 9 on demand: listens ONLY on 127.0.0.1, random port, token from env `P9D_TOKEN` (never in the command line; >=16 chars or the exe exits 2), ONE port speaks HTTP proxy (CONNECT + absolute URI) and SOCKS5, Basic/user-password auth (password = token, else 407), `GET /p9direct/health` , prints `{"ready":true,"port":N}` on stdout and exits when stdin closes (no orphan). `DirectHelper` restarts a crashed helper (<= 5 per 2 min, exponential delay). Chromium cannot do SOCKS5 auth, so the sessions use it as an **HTTP proxy**; the credentials are answered silently via the app `login` event ( `direct.owns(host,port)` ). DNS is resolved by the helper (the Windows resolver) — a caveat: DNS then also goes outside the VPN tunnel only if ExpressVPN excludes the exe.
   - Granularity = a session: the **Main session** (profile setting `bypassMain` ), one **account container** ( `container.bypass` ), or one **private window** ( `ctx.privBypass` ). UI: toolbar toggle `#bypass` (applies to the active tab's container/window), the location menu ("Outside the VPN"), the Accounts page pill, the extension popup, `p9ctl bypass` .
   - **Precedence**: a Panel 9 proxy / PIA region of the profile/container ALWAYS wins; the toggle is then shown dimmed and labelled "ignored: is used" ( `resolveProxy` -> `bypassIgnored` ). Chip tooltip names the route ("Outside VPN — Panel9 Direct").
   - **Fail-safe**: if the helper is down, that session is BLOCKED ( `proxyx.BLOCK_CONFIG` , dead-end) by default, or falls back to the system/VPN route when the user set `fail:'vpn'` ( `userData/expressvpn.json` ). The exit chip then says "Blocked". WebRTC IP handling is pinned for bypassed sessions ( `wantsWebRtcPin` ).
   - **Verification** ( `verifyBypass` ): the IP seen through the helper (session `p9-vpn-verify-helper` ) vs through the normal path ( `p9-vpn-verify-plain` ): different -> `ok` ; same while ExpressVPN is connected -> `leak` ("Panel9 Direct is not excluded", with the exact steps); same while not connected -> `unknown` (says so); no answer -> `down` . Re-run on every VPN state change.
   - **Split-tunnel rule helper**: `split check` reads `get split-app` (lines `bypass:<path>` / `vpn:<path>` ); `split add` is EXPERIMENTAL ( `set split-app bypass:<path>` , 4 spellings because of a reported ':' quirk) and is always

verified by reading the list back; if the CLI replaces the list instead of adding, the old entries are restored and it reports that it cannot add safely. The extension also has a click-by-click guide (ExpressVPN -> Settings -> Split tunneling -> "Do not allow selected apps to use the VPN").

- **Extension** ( `/workspace/panel9-expressvpn` , id regex `EVPN_ID_RE` in `src/extbridge.js` ): popup tabs Locations (searchable, flags) / Outside VPN / Setup. Bridge op `vpn` with whitelisted fields; the helper token never leaves main (snapshots/logs/control output exclude it).
- **p9ctl**: `vpn status|connect [location]|disconnect|regions [filter]|split check|add|verify` , `bypass on|off [--window W] [--container X]|status|verify` . Private windows can be switched per window only. No focus is taken (asserted).
- Tests: `test:helper` ( `go test` : forward + 407, CONNECT, SOCKS5, health), `test:vpn` (mock CLI `test/mock-expressvpnctl.js` + the REAL helper binary on Linux; routes proven by source address: the helper dials from 127.0.0.21 via `P9D_SOURCE` , direct traffic from 127.0.0.1), `test:ctlvpn` (real relay + p9ctl). `P9_PROXY_LOOPBACK` makes 127.0.0.1 go through proxies in tests.
- **Confirmed vs untested**: everything above runs against the Linux build of the same Go source and a mock CLI. NOT verified on real Windows / real ExpressVPN: the real `expressvpnctl.exe` output formats, the admin requirement in practice, whether ExpressVPN honours the split-app entry for this exe, Windows execution/AV/SmartScreen of the unsigned exe, the NSIS/portable packaging on Windows.

## 2.3.5: Grok Bot control never steals the OS focus

**Problem.** The user works in one Panel 9 window while another agent controls a tab in a second window; every control command pulled focus to the controlled window ( `wc.focus()` before mouse/type/key, `ctx.win.focus()` in `open` / `switch` , `activateTab()` -> `webContents.focus()` , `focusOrOpen` /new windows shown active, popped-out web-app `win.show()` ). A desktop only has ONE focused window at a time (OS rule), so the fix is not "focus two windows" but "never touch focus at all": the controlled page must work while its window is in the background. **Rule (enforced by `test:ctlnofocus` ).** No control command calls `BrowserWindow/BaseWindow.focus/show/moveTop/restore` or `webContents.focus()` unless the caller passed an explicit `--focus` . The only focus-capable code left in `src/control.js` is `_raise(ctx)` (explicit `--focus` ) and `_focusIfActive(t)` (fallback input path, and only when that window already IS the focused one).

- **Input through the Chrome DevTools Protocol** ( `Control._cdp/_viaCdp/_mouse/_insert/_press/_clearField` ). `wc.debugger.attach('1.3')` once per webContents (state in a `WeakMap` ; re-attached automatically if DevTools, `emulation.js` (mobile -> desktop switch detaches) or anything else detached it; `detach` listener resets the flag). Then `Emulation.setFocusEmulationEnabled({enabled:true})` (the page behaves as focused: `document.hasFocus()` , `:focus` , caret) and `Input.dispatchMouseEvent` (mouseMoved, mousePressed, mouseReleased with `clickCount` / `buttons` : `isTrusted` true - Google sign-in still works), `Input.insertText` (typing), `Input.dispatchKeyEvent` (keys: table `NAMED_KEYS` ; modifiers bitmask; `text` for characters and Enter; Ctrl/Cmd+A/C/X/V/Z/Y carry the editing `commands` so select-all works through CDP). Coordinates are CSS pixels (what `elements` / `_locate` / `_clickText` already return), so no more view-pixel scaling on this path.
- **Mobile-emulated tabs** (touch emulation on): a left click is a tap ( `Input.dispatchTouchEvent` touchStart/touchEnd; CDP mouse presses are never acked in a touch-emulated page and would hang); right click there uses the fallback. Every CDP send has a 10 s cap.
- **Fallback.** If the debugger cannot attach (another client, `P9_CTL_NO_CDP=1` ) or a call fails *before anything was dispatched*, the old `sendInputEvent` / `wc.insertText` path runs, but `wc.focus()` is only called when the window is already the focused window ( `_focusIfActive` ). After the first dispatched event an error is thrown instead (no double clicks).
- **Painting while in the background.** `_prep(t)` : `wc.setBackgroundThrottling(false)` on every controlled view. While control is enabled at startup ( `control.json` ), `main.js` also starts Chromium with `--disable-features=CalculateNativeWinOcclusion --disable-backgrounding-occluded-windows` so a window fully covered by the user's window keeps painting (needs a restart after first enabling control). `screenshot` races `capturePage` against 10 s and reports an empty/never-rendered frame (minimized window: use `p9ctl switch --focus` ).
- **Tabs inside the window.** `_show()` only makes the tab the visible one INSIDE its own window ( `activateTab(ctx, id, {focus:false})` : no `webContents.focus()` , no address-bar focus, no raise) because a hidden view does not paint; `close` uses `closeTab(ctx,id,{focus:false})` (neighbour activation + a replacement tab never focus).
- **open / switch / profile / add-account / webapp open** are no-focus by default and take `--focus` (p9ctl help) to additionally raise + focus the window (the old behaviour, `_raise` : restore, show, focus). `open` creates the tab with `createTab(..., {noFocus:true})` and activates it inside its window; when the profile has no window yet `openWindow(pid,`

`{inactive:true})` creates it hidden and `showInactive()` s it (and does not make it the app's "current" window). `switch` = in-window tab switch. `profile.switch` = `focusOrOpen(pid, {focus:false})` . `add-account` = `addGoogleAccount(ctx, {verify:true, noFocus:true})` + in-window activation. `webapp open` = `wa.open(ctx,id,{toggle:false, inactive:true})` : a popped-out web-app `BaseWindow` is created `show:false` and `showInactive()` d (docked sidebars never take focus).

- **Old Panel 9 (< 2.3.5)** ignores `--focus` (it always raised); nothing else on the wire changed. Token, E2E encryption, replay protection and the activity log are unchanged (no new fields are logged).
- **Preserved from 2.3.3/2.3.4** (untouched): trusted clicks for Google ( `_clickText` -> `_mouse` , now CDP), updater PowerShell wrapper + `quitForUpdate` , hardened `add-account` , `--webapp` control ( `webapp.exec` ), log privacy.
- **Tests.** `npm run test:ctlnofocus` ( `test/control-nofocus-checks.js` ): a second Panel 9 window for another profile is created by control `open` and then driven (click/double/type --clear/--submit/key/scroll/text/elements/eval/wait/shot, background tab, mobile tab, `close` , `add-account` ) while window 1 holds the focus; `BaseWindow.prototype.focus/show/showInactive/moveTop/restore/minimize` are spied per window, `webContents.focus` is spied on every view of the controlled window, and the window's own `focus` events are counted: all stay at zero (only `showInactive` is allowed) and the focused window is unchanged. Also: click `isTrusted` true, `document.hasFocus()` true in the unfocused window, typed text/keys land, PNG is a real (non-blank) render, debugger detach -> re-attach, debugger attach conflict -> `sendInputEvent` fallback (with and without a focused window), explicit `--focus` raises, `p9ctl help` , and a static guard on `control.js` . `test:ctlwebapp` additionally covers a popped-out web-app window (inactive show, page commands never focus it).
- **Not covered / known limits.** The Linux box has no window manager, so "the OS really kept the focus" is proven by the spies + focus events + `isFocused()` , not by watching a Windows desktop. A page that itself calls `window.focus()` in a background window is Chromium's business, not control's. A minimized window cannot paint ( `shot` reports it). Right-click opens Panel 9's native context menu in the controlled window (as before). A CDP debugger stays attached to controlled tabs (invisible to pages, but some anti-bot systems can detect DevTools-style attachment; `P9_CTL_NO_CDP=1` restores the old path).
- **Ship.** 2.3.5 portable ( `--platform windows` ) and setup ( `--platform windows-setup` ); `p9ctl` is synced to `/workspace/panel9-control/p9ctl` .

## 2.3.4: Grok Bot control of web-app extensions ( `webapps` , `webapp open|close` , `--webapp` )

**Web-app extensions (background).** An extension whose manifest has a `panel9_webapp` block (Telegram, TechiePoolChat, YT Panel) is not a popup page and not a tab: `src/webapps.js` ( `WebApps` , instance `wa` in `main.js` ) hosts it in its own `WebContentsView` ( `i.content` ) plus a 36 px header view and a resize grip, in the persistent partition `persist:p9wa-<profile>-<extId>` . One instance per window+extension ( `wa.get(ctx,id)` ); docked as a right sidebar (only one docked at a time, `show()` hides the previous) or popped out into a ~400x650 `BaseWindow` ; closing only hides it (it keeps running, unread count polled every 3 s). Clicking its toolbar icon = `wa.open(ctx,id)` (toggle). Before 2.3.4 the control layer only knew tabs ( `_tabs()/_find()` ), so these views were unreachable. **What 2.3.4 adds ( `src/control.js` , `src/webapps.js` , `tools/control/p9ctl` ).**

- `webapps` (alias `webapp.list` ): `WebApps.list(ctx)` = every installed **enabled** web-app extension: `{id, name, created, open, docked, loaded, url, unread}` (+ `profile` , `window` ). `open` = visible now, `created` = a view exists (running, maybe hidden), `url` = the live URL when created, else the configured one.
- `webapp.open` {webapp}: `wa.open(ctx,id,{toggle:false})` - creates the view if needed and shows it exactly like the toolbar icon (idempotent, never toggles it closed), then waits up to 15 s for the first load. `webapp.close` : hides it (keeps running).
- `--webapp <id|name>` = args `{webapp}` instead of `{tab}` on **text, html, elements, eval, click, type, key, scroll, wait, screenshot** ( `WEBAPP_CMDS` ). `Control._find(id, a)` returns `_findWebapp(a)` when `a.webapp` is set: a pseudo tab `{id:'webapp:<ext>', webapp:i, view:i.content, mode:'desktop', containerId:'default'}` so every existing handler (isolated-world JS, `_locate` , trusted `_mouse` via `sendInputEvent` , `insertText` , `key` , `capturePage` ) runs unchanged on the extension's `webContents` . `_show()` for a webapp = `wa.show(i)` when hidden (input/screenshot commands need a visible view; read-only commands - text/html/elements/wait/eval/scroll - work on a hidden running view). A web-app that was never started gives `X has not been opened yet: run "p9ctl webapp open <id>"` (page commands never create views behind the user's back).

- **Name matching** ( `Control.matchWebapp` , pure, unit-tested): case-insensitive; exact id > exact name > id without `local-` > prefix of name/id > substring. Ambiguity at the best tier is an error that lists the candidates; unknown names list what is installed. The window is the focused allowed one (or `--profile` ).
- **Rules.** `--webapp` with `--tab` is refused; with any other command ( `go` , `close` , `switch` , `open` , `add-account` , `signin-google` , ...) it is refused instead of silently acting on a tab. Private windows and profiles that are not on the allow list are invisible ( `_webappCtxs` ). `eval` still needs the user's "script execution" switch; password inputs are still stripped from `html` ; the input pause / password-manager block ( `lastInputAt` ) applies.
- **Wire format / old-build safety.** p9ctl sends `--webapp` page commands wrapped as `webapp.exec {webapp, cmd, args}` (unwrapped at the top of `Control.run` , inner cmd restricted to `WEBAPP_CMDS` ). A Panel 9 older than 2.3.4 has no such command and answers `unknown command: webapp.exec` - it would otherwise have ignored an unknown `webapp` argument and run the command on the active TAB. `p9ctl` adds "update Panel 9 to 2.3.4" to that error. `p9ctl run` users can send `webapp.exec` lines too.
- **Security / log.** Same token, E2E AES-GCM and replay protection (nothing changed in the transport). Activity log ( `control-log.jsonl` , shown on `panel9://control` ) records `{e: <command>, webapp: <extension name>}` and nothing else for these commands - no host, profile, character count, typed text, selector or page content (the 'type' length is dropped too). Error messages only echo the (clipped) extension name asked for.
- **p9ctl.** `p9ctl webapps | webapp open|close NAME|ID | <page cmd> --webapp NAME` (see `p9ctl help` ). Example (Telegram): `p9ctl webapp open telegram; p9ctl text --webapp telegram --selector '.chatlist'; p9ctl type "alice" --webapp telegram --selector 'input[type=search]' --clear; p9ctl click --webapp telegram --index 7; p9ctl type "hello" --webapp telegram --selector '.input-message-input' --submit` . Selectors for Telegram Web K must be read from `p9ctl elements --webapp telegram / html` ; they are Telegram's, not ours, and can change.
- **Tests.** `npm run test:ctlwebapp` ( `test/control-webapp-checks.js` ): real relay + real p9ctl + fixture "Telegram"/"TechiePoolChat" web-app extensions (chat list, search box, contenteditable composer): name matching (unit), listing, open/idempotent/unknown/ambiguous, text/html/elements/click(isTrusted)/type/--submit/key/scroll/wait/shot/eval(switch), --tab+--webapp and wrong-command refusal, `webapp.exec` raw protocol, close/hidden reads/auto-show, one sidebar at a time, allow list, private windows, wrong token, activity-log privacy. Not tested: the real web.telegram.org (needs the user's login).
- **Facebook web-app extension (new store item `panel9-facebook` 1.0.0, folder `/workspace/panel9-facebook` ).** Built exactly like Telegram ( `panel9_webapp` block: `url` https://www.facebook.com/, not configurable, width 420 x 700, `allowHosts` facebook.com / fbcdn.net / messenger.com / fb.com, default `(N)` title unread script, `notifyFallback: true` ; matte-black rounded tile + Facebook-blue disc + white "f" icons generated by PIL; README + ARCHITECTURE.md; zip = manifest.json + icons). Like Telegram/TechiePoolChat it is NOT baked into the Panel 9 exe: it is a store extension (type `extension` , target `panel-9` , platform `any` ) the user installs from Extensions -> Store; the control name is its manifest name, so `p9ctl webapp open facebook` and `--webapp facebook` work once installed. Publish: `publish.sh panel9-facebook 1.0.0 /workspace/panel9-facebook-1.0.0.zip "notes" --type extension --target panel-9 --platform any --name Facebook ... --icon store-icon-512.png --source /workspace/panel9-facebook --arch /workspace/panel9-facebook/ARCHITECTURE.md` . Verified: manifest block, listing and name resolution ( `test:ctlwebapp` ); NOT verified: logging in to the real facebook.com (needs the user's account; Facebook may show checkpoints/captchas).
- **Preserved from 2.3.3** (untouched): updater PowerShell wrapper + `quitForUpdate` , `signin-google` trusted clicks, hardened `add-account` .
- **Ship.** 2.3.4 portable ( `--platform windows` ) and setup ( `--platform windows-setup` ); `p9ctl` is synced to `/workspace/panel9-control/p9ctl` .

## 2.3.3: trusted-click Google sign-in, hardened add-account, smooth installed update

- **signin-google.** 2.3.2 pressed Google's Next with DOM `.click()` , which Google ignores. Buttons (Next/Continue/Not now/Skip) are now found by text ( `Control._clickText` : smallest visible, enabled match → nearest button → bounding-box centre) and pressed with the same trusted `_mouse` (sendInputEvent) as `p9ctl click` . Fields: Ctrl+A/Backspace + `insertText` , then **Enter**; only if the page did not react within 2-3 s is Next clicked. Waits are 100 ms polls on URL/text change ( `waitChange` ), not fixed sleeps. Passkey enrolment, selfie and other extra prompts always get **Not now**. Activity log: command + email domain only.
- **add-account.** `addGoogleAccount(ctx, { verify: true })` (control only) refreshes a spare "New Google account" container's cookies before reusing it; a signed-in spare is skipped (a new container is created instead). It also returns the add-

account `url`. The command always activates the returned tab, waits up to ~12 s for it to reach accounts.google.com (loads `url` itself when the tab sits on `panel9://newtab`/blank), closes stray blank tabs it caused, and reports `onGoogle`.

- **Installed update.** `applyUpdate(variant 'installed')` no longer spawns the setup directly: it writes a PowerShell wrapper (`updater.buildSetupScript`) that (1) waits for the calling app PID, (2) waits until no `Panel 9.exe` under `%LOCALAPPDATA%\Programs\Panel 9` runs (force-closing leftovers after 8 s, gives up at 45 s), (3) stops a hung earlier `*setup*.exe` from the updates folder (named mutex `Panel9UpdateSetupWrapper` stops a second wrapper from stacking), (4) runs `setup /S --updated` (NSIS `runAfterFinish` relaunches the new version) with a 5 min cap (then kills it and restarts the old app). Log: `%AppData%\Panel 9\updates\update-setup.log`. `main.js quitForUpdate()` = `app.quit()` then `app.exit(0)` after 4 s if something blocks quitting. Portable swap script unchanged.
- **Ship.** 2.3.3 portable + setup. `p9ctl` unchanged (no new verbs).

## 2.3.2: fast Add-account + Google sign-in control commands

- **Why.** Driving Google sign-in with many small `p9ctl` round-trips (click / type / wait / screenshot) was the real lag, not the relay. Each step paid a full settle+broker hop.
- **Commands.** `add-account` calls the same `addGoogleAccount` path as Panel 9's YouTube "Add account" row (fresh empty container when the current one is signed in) and returns the new tab. `signin-google --email --password [--totp SECRET|--code N]` drives email → captcha click → password → TOTP (HMAC-SHA1 from the base32 secret, or a supplied 6-digit code) → skip passkey/selfie, in ONE round trip on the Panel 9 side. Activity log records the command name and email domain only — never the password or TOTP secret.
- **Wiring.** `src/control.js` cmds + `addGoogleAccount` passed into `new Control({...})` from `src/main.js`. `tools/control/p9ctl` (and `/workspace/panel9-control/p9ctl`) accept the new verbs; sign-in uses a 130 s client timeout.
- **Ship.** Bump to 2.3.2; publish portable + setup. Live Panel 9 must update before Grok Bot can use the fast path.

## 2.3.1: control reliability (page blink, banner flapping, blank view)

- **What went wrong in 2.3.0.** (a) Every `p9ctl <cmd>` was its own relay connection, so Panel 9 saw "Grok Bot attached → command → detached" in ~500 ms and the "controlling" banner (which is part of the chrome height) showed/hid and re-laid-out the content view on every command (32 flips for 20 commands in a desktop tab, 40 in a mobile tab, measured on the real 2.3.0 build). (b) `panel9://control` rebuilt its whole DOM every 4 s and on each status event, and `.wrap` has an entry animation (`rise`), so the page re-played that animation each time ("blinks"). (c) With the relay unreachable the client retried every 1.5 s → 30 s with no jitter and pushed a status change on every attempt.
- **Fixes.** `src/control.js`: retry schedule `Control.retryDelay` = $2\,s \cdot 2^n$ capped at 60 s with ±20 % jitter; state `waiting` (stable) with `attempt` / `retryAt` in the status; connection events go through `_notify()` (coalesced 250 ms, only when the visible state really changed). `src/main.js` `syncControlBanners`: the banner shows at once and is **sticky for 60 s** after the last attachment (`P9_BANNER_HOLD_MS` overrides, tests only); switching control off hides it immediately. `layout()` ignores a degenerate (< 120 px) window rectangle. **View watchdog** (`viewWatchdog`, every 2.5 s): if the active tab's view is detached, hidden, zero-size, off its expected rectangle or its renderer crashed, it is restored (`viewProblem` classifies; `watchdogStats` counts). `src/pages/control.js`: render once, then `refresh()` only re-renders when a visible/editable value changed (signature) and otherwise patches the status chip text ("Waiting for relay, retrying in Ns") in place; scroll position is kept. `p9.css` / `p9.js`: `.wrap`'s entry animation plays once per page load (`body.settled`). The banner icon no longer pulses forever.
- **p9ctl broker** (`tools/control/p9ctl`): the first invocation starts a background broker that owns ONE relay connection; later invocations talk to it over a user-private socket (unix socket / Windows named pipe `\\.\pipe\p9ctl-<id>`) after an HMAC proof of the token; it exits after 120 s idle (`P9CTL_BROKER_IDLE_MS`), `P9CTL_NO_BROKER=1` disables it, `p9ctl run` always uses one direct connection. Connection errors exit with code 3.
- **Tests.** `test/control-flicker-checks.js` (relay down: no rebuilds, no animation replay, backoff, banner) and `test/control-burst-checks.js` (20 rapid p9ctl commands, real relay, shots `shots/control-burst-*.png`, watchdog, broker). The blank view seen on Windows could NOT be reproduced on Linux/xvfb (the flapping could); the watchdog and the degenerate-rectangle guard are the safety net for it.

## 2.3.1: Add account = fresh empty container; Sign out; switcher

- **Why "Add account" loaded the signed-in account.** Under one-account-per-container the decision "does this container already have an account?" used the cached e-mail count, which is empty/stale right after launch or for a container signed in elsewhere; the navigation then went on inside the signed-in session (Google showed the existing account). Now

`knownSigned(ctx,cid)` = cached count **or** `ctx.googleSigned` (kept by `trackGoogleSignIn`, which watches the container's Google auth cookies `SID/HSID/SSID/APISID/SAPISID/__Secure-*PSID/LSID`, via `accounts.googleCookies` - google.com plus the test hosts) and every entry point falls back to an async `accounts.detect` (cookies + ListAccounts) when the sync answer is "empty".

- **Entry points (all end in `overflowSignIn` / `addGoogleAccount`):** (1) YouTube's own Add account link: the preload (`pages/preload.js`) captures a trusted click on any `accounts.google.com/AddSession` link and sends `p9:add-account` (also routed by `will-navigate` / `will-redirect` / `did-start-navigation`, so scripted navigations and account-menu redirects are caught, desktop and mobile YouTube); (2) `window.open` / popup windows: `setWindowOpenHandler` (tabs and popups) and `attachPopup`'s `will-navigate` call `maybeOverflowAddAccount(..., defer=true)` which denies the popup and decides asynchronously (signed in -> fresh container, empty -> opens a tab in place); (3) ServiceLogin / account-chooser navigations started from a non-Google page of a container that has an account (`fromNonGoogle`, `isSignInUrl`) go through `lateOverflowCheck`; (4) the injected card row `[data-p9-add-account]` (trusted click only) and the switcher button `#addgoogle` call `accounts.add` -> `addGoogleAccount`.
- **Fresh container:** `accounts.overflowTarget` (reuses a still-empty "New Google account" container, otherwise creates one with `auto:true`), then a tab on `accounts.addAccountUrl(ctx,cid,next,true)` = `https://accounts.google.com/AddSession?service=youtube&continue=<youtube>` (`p9fresh` marks the tab so it is not intercepted again). The container has its own session partition, so no cookie is shared and the e-mail field is empty. After sign-in the existing auto-naming renames the container to the e-mail and it shows in the switcher. In shared mode (opt-in "Up to 10") the old behaviour is unchanged.
- **Sign out.** Panel 9 core never hides YouTube's Sign out (verified by test); the only hider is the yt-focus extension, now with a **Hide Sign out** switch (1.2.0, default ON). Panel 9's switcher (`panel9://accounts`) has per account **Sign out** (`containers.signOut`: erases the container's session, keeps the container) and **Remove** (`containers.remove`: deletes the container; not offered for the built-in Main container), both with a confirmation box.
- **Look.** Injected card: avatar stack, "All accounts (Panel 9)..." and "+ Add account" rows with icons, hover/active states, 120-180 ms transitions, YouTube's own colour variables (follows light/dark), one card per popup (nested menus no longer duplicate it). Switcher: avatar stack, dashed "+ Add account" pill (icon rotates on hover), icon buttons with hover colours. `p9.css` section "account switcher (2.3.1)".
- **Tests.** `npm run test:addaccount` (`test/addaccount-checks.js`: fixture server for Google, stale cache + cookies, native link, injected card, popup, ServiceLogin, programmatic navigation, mobile YouTube, switcher button and UI, yt-focus toggle; shots `shots/addaccount-*.png`) and `npm run test:addaccountlive` (real accounts.google.com, needs internet: empty e-mail field, no Google cookies, `shots/addaccount-live-google-empty-email.png`). Run Electron tests with `--no-sandbox` (the npm scripts do): on some boxes the renderer of heavy pages crashes without it (`/dev/shm` shared-memory errors).

## 2.3.6 — Save-password popup sizes to its content

`src/pwbubble/` (a WebContentsView over the page): the popup used fixed heights and a single non-wrapping button row, so on Windows (different font metrics, 125/150 % scaling or zoom) "Not now / Never for this site / Save" were clipped. Now `bubble.js` reports its real height with a ResizeObserver (`pwb:size`), `pwglue.js` `place()` sizes the view to it (multiplied by the view's zoom factor, width = 380 DIP x zoom, capped to the window), buttons are `nowrap` but the row wraps (stacks below 300 CSS px), inputs are `border-box`, long host/user names wrap or ellipsize. Test: `test:pwbubble` renders it in Matte dark/light and all six store themes at 100/125/150 % zoom for save/update/locked/first-time-setup and in a 360 px window, asserting every button lies fully inside the popup (`P9_SHOTS_DIR` keeps screenshots).

## 2.3.6 — Lounge and the editors (`panel9://lounge`, `src/lounge.js`, `src/loungeglue.js`, `src/pages/{lounge,text,code,csv,xml,pdf}.*`)

- **Lounge** is a per-profile file space under `userData/lounge/<profileId>/files` (not available in private windows). `src/lounge.js` is pure Node: paths are always relative and `/`-separated, validated (no `..`, no reserved Windows names, no dot-names, no symlinks, depth <= 12, name <= 180) and re-checked against the root; 512 MB per file; chunked upload sessions (10 min expiry) write to `lounge/<id>/tmp` and are renamed into place.
- **Editors** are internal pages opened as normal tabs: Text (CodeMirror, encoding + EOL aware), Code (CodeMirror 6, language modes, Prettier, live HTML/CSS/JS preview in a sandboxed server on 127.0.0.1), CSV (virtualised grid, <= 160 DOM rows, delimiter/quote preserving), XML (tree + source, validation, XPath, pretty/minify), PDF (pdf.js viewer; pdf-lib for annotations,

page rotate/delete/reorder/merge/split, form filling). Everything is bundled by `tools/build-vendor.js` into `src/vendor/` — no CDN, strict CSP with nonces ( `handlePanel9` sets a per-response nonce; CodeMirror gets it through `EditorView.cspNonce` ).

- **Content handoff**: pages never get file paths. Content fetched for an editor lives in the main-process *stash* (24 entries / 256 MB / 1 h) and is referenced by an opaque id; saving goes through `lounge.*` api9 calls.
- **Context menus** ( `ctxmenu.js` ): "Open in Code / PDF / CSV / XML / Text Editor", "Save to Lounge", "Edit page source"; file rows in the Lounge page and the Downloads page have Open / Open with / Download / Rename / Move / Delete. Setting `openDownloadsInPanel9` (default off) opens finished downloads of editable types in the matching editor.
- **p9ctl**: `p9ctl lounge|files list|stat|search|read|write|upload|download|mkdir|rename|delete|open` ( `src/control.js` `lounge.*` ). Binary-safe, base64 chunks of 1 MB (relay limit 8 MB), `begin/chunk/finish/abort` for large files, never focuses or raises a window unless `--focus` , the token never appears in output or in the activity log.
- **Tests**: `npm run test:lounge` (pages, every editor, menus, CSP, preview sandbox) and `npm run test:ctllounge` (real relay + p9ctl, large/binary files, no focus). Both are part of `test:modules` .

## 2.3.6 — Web-app view guard (blank / stale web-app windows) ( `src/viewguard.js` , `src/webapp/state.*` , `src/webapps.js` )

Root causes found for "the web-app window is blank / shows a stale frame" and the fix for each:

1. **Load before bounds** — docked apps called `loadURL` before the layout gave the view its size, so Chromium could start at 0x0 and never produce a first frame. Now `show()` lays out first, then loads.
2. **`did-fail-load` / `render-process-gone` were stored and ignored** — nothing was retried and nothing was shown. Now: retry 3 times with back-off (1.5 s, 4 s, 10 s) behind a "Reconnecting..." card, then an error card with the reason in plain language, the address and **Retry** / **Open in browser** buttons. Renderer crashes restart automatically (but stop after 3 in a minute and show the card). A renderer that is unresponsive for 6 s is force-restarted.
3. **Stale compositor frame** after minimise/restore, hide/show, maximise, fullscreen, focus or a monitor / DPI change — `repaint()` invalidates and nudges the bounds by 1 px (and restores the exact bounds 30 ms later). Triggered on all of those events for docked and popped-out apps, and on `screen` display events.
4. **No paint verification** — after `did-finish-load` the watchdog checks the DOM (text, media, controls, viewport size) and a sampled compositor frame; if blank: repaint -> reload -> recreate the WebContentsView -> error card. Hidden apps are never judged (checked when shown).
5. **White flash / empty rectangle** — views start with the theme's matte background and a skeleton overlay (own view above the content, same bounds) until the page is verified painted.

- Every recovery is recorded in memory ( `webapps.guardLog()` ) and appended as JSON lines to `userData/webapp-recovery.log` (rotated at 200 KB; app name, kind, short reason — no page content). Counters: `webapps.guardStats()` .
- `P9_GUARD_SCALE` (env, tests only) scales every guard timing. Test: `npm run test:viewguard` (fixture server: healthy, slow, refused connection + recovery, blank-then-fine, always-blank, crash, hang, crash loop, restore/display change, popped-out window, hidden app, diagnostics).

## 2.3.6 — Not in this release (moved to 2.3.7)

Panel 9 Agent (Gemini assistant, local activity memory, tools), the Grok Bot bridge channel + webhook and the `panel9-grokbot` extension were scoped for 2.3.6 but are **not shipped**: their foundations (SQLite/FTS5 store, Gemini client) were prototyped and are parked outside the source tree so nothing half-built is in the installer.

## 2.3.7 — Updater helper must outlive the app ( `src/launcher.js` )

**Symptom (real PC, installed 2.3.4 -> 2.3.6):** the update downloaded and verified, the PowerShell wrapper `%TEMP%\p9-update-setup-*.ps1` was spawned three times (three update prompts in a row), but none ran: no `update-setup.log` (not even "wrapper start"), the `.ps1` was never deleted (its `finally` never ran), the old version relaunched and asked again. Running the same `powershell -ExecutionPolicy Bypass -File` by hand worked and all execution policies were Undefined.

**Root cause (most probable; not reproducible on the Linux build box):** the helper was a plain `child_process.spawn(..., {detached:true})` child of the Electron process and the app quit 300 ms later. `detached` only sets `DETACHED_PROCESS | CREATE_NEW_PROCESS_GROUP` ; it does **not** break away from a Windows job object. A process launched from Explorer / a launcher / a security product can live inside a job with `JOB_OBJECT_LIMIT_KILL_ON_JOB_CLOSE` , and every child spawned from it inherits the

job, so when the app (and the job) ended the half-started PowerShell (~1 s startup) was killed before its first statement. Nothing ever checked that the helper had started. (A product blocking `powershell.exe` from `%TEMP%` looks identical; `update-launch.log` now tells the cases apart.) The script itself was checked: it parses cleanly and its first statements run under real PowerShell.

**Fix:**

1. `launchHelper()` starts the helper through a process that is not the app's child and not in its job: **(1) WMI** `Win32_Process.Create` (parent = `WmiPrvSE`), **(2) one-shot Task Scheduler task** (`schtasks /Create ... /SC ONCE`, `/Run`, `/Delete`, current user, no admin), **(3) plain detached spawn**.
2. The helper writes a **handshake file as its very first action** (ps1: first statement, before the mutex and the log; bat: first command). The app polls it (200 ms) and **quits only after it appears**. No handshake in 10 s (or the launch request itself fails) -> next method. Failed requests fail fast.
3. All methods failed -> installed copy: **the setup opens visibly** (`shell.openPath`, app stays open, dialog explains); portable copy: the app stays open, the new exe is kept beside the old one and revealed.
4. **Loop guard** (`update-attempt.json` in userData): two silent attempts at the same version without the version changing -> the third goes straight to the visible installer. Cleared on the first start of the target version.
5. Same launcher for the portable swap `.bat`, the legacy rename `.bat`, and the portable -> installed migration (new `buildRelaunchScript`: waits for the portable app's PID, then starts the installed exe, which previously was spawned directly and also hit the single-instance lock held by the still-running portable app). Swap/rename/relaunch `.bat`s take a lock directory so a slow first method plus a fallback cannot run twice; stale locks (> 10 min) are cleared.
6. `.ps1` is written with a UTF-8 BOM (Windows PowerShell 5.1 reads BOM-less files as ANSI and garbles non-ASCII user names); parent-exit wait raised 15 -> 30 s; leftover `p9-update-*.ps1/.bat/.started` older than 1 h are swept at startup.
7. Every attempt is appended to `userData/update-launch.log` (method, handshake received or the exact failure). Tests: `test/launcher-checks.js` (`npm run test:launcher`, virtual clock): handshake success, 10 s timeout -> next method, fast failure, all-fail, scheduled-task cleanup, real detached process + real file handshake, loop guard, handshake is the first statement of every script, `applyUpdate` visible/portable-manual fallbacks, relaunch script. `applyUpdate` is now `async`. **Not verified:** nothing here has run on real Windows (no Windows box); only the logic with fake spawn/exec, a real POSIX process for the handshake, and the wrapper under PowerShell 7 on Linux.

## 2.3.7 — Panel 9 Agent, Grok channel, `panel9-grokbot` (`src/agent/*`, `src/pages/agent.*`, `src/extbridge.js`)

- **Layout.** `src/agent/` is pure Node + `node:sqlite` (FTS5), no Electron imports except in `glue.js`: `db.js` (schema, FTS, retention, forget, kv), `memory.js` (layers, hybrid recall), `privacy.js` (what may be recorded: URL cleaning, sensitive pages, excludes), `local.js` (instant local intents), `tools.js` (function declarations, validation, confirmation rules), `gemini.js` (client), `agent.js` (tool loop), `grok.js` (`GrokChannel`), `recorder.js` (event capture), `glue.js` (per-profile bundle: wires Panel 9 APIs, settings, the pane and the control API), `agenthost.js` (tool implementations against the browser). The pane/page is `panel9://agent` (`src/pages/agent.{html,css,js}`), opened in a right-hand side pane (Ctrl+Shift+Space, toolbar sparkle button, Agent menu) with a width grip, and from the new-tab greeting card.
- **Memory layers (per profile, `userData/agent/<profile>.db`).** (1) raw `events` (visit, tab_open, download, lounge, ext, ...) — never recorded in private/incognito windows, when paused, or for per-site excluded hosts (excluding also erases that host's history); (2) `episodes` — LLM- or heuristic summaries of event runs; (3) `facts` — semantic facts with Gemini embeddings (`gemini-embedding-001`; chat uses `gemini-2.5-flash-lite` / `gemini-2.5-flash`), pin/delete; (4) `skills` — procedural, recorded from a run ("Save as skill", "learn this as X") or taught in chat; feedback (Good / Not right) adjusts use counts and creates corrections. Recall = hybrid FTS5 BM25 + cosine, merged; all searches are local.
- **Speed.** Greetings, time/date, tab counts, name changes, settings toggles and recall answers are local (no network); open-ended questions stream from Gemini over SSE; a warm cache of the last tabs/recents is kept in memory; summarising/embedding runs in idle background steps with a queue and quota cool-down. No worker process: a 50k-event search measures well under the budget (see `test/scale-bench.js`), so the main-process design stays.
- **Gemini key.** Stored with `safeStorage` (Windows DPAPI) in the profile's kv, never returned to the page (write-only, only "saved" state), sent only in the `x-goog-api-key` header, never logged. Without a key the Agent still greets, recalls, searches history and runs local commands, and explains how to add a key.
- **Control.** Gemini function calling over a tool registry: tabs (list/open/switch/close/restore session), history search, Lounge files, settings, downloads, screenshots, password manager (names/sites only — secrets are never revealed to the model), VPN/bypass, extensions (list/open popup/pin), `ask_grok`. Destructive actions and sending messages return a confirmation card first.

- **Grok channel ( `grok.js` ).** Messages are stored in the agent database (direction `in` / `out` ), so they work without the webhook: Grok polls with `p9ctl grok inbox|wait` . If a webhook URL is set and enabled, every message from the user/Agent triggers a POST: `Authorization: Bearer <key>` (header name and scheme configurable), `content-type: application/json` , `user-agent: Panel9/<version>` , `x-panel9-event: message|test` , `x-panel9-delivery: <id>` ; JSON body `{event:'panel9.message', id, ts, kind, text, reason, context, agent, app, version, reply:{via:'p9ctl', command, inbox, wait}, unread}` . Retry with backoff 2/6/20/60/180 s on network/408/425/429/5xx; 401/403/404 and other 4xx fail permanently (shown with a Retry button); https only (http for localhost), redirects refused. The key is encrypted at rest and never logged or echoed (preview shows `Bearer ••••` ). Presence = a `grok.*` call within 90 s.
- **p9ctl.** `p9ctl grok status|inbox [--all] [--peek] [--limit N]|wait [--timeout S] [--peek] [--loop]|reply "text" [--to ID] [--from NAME]|ack ID` and `p9ctl agent status|ask "request"|recall "what"|skills|name [NEW]|open` ( `control.js` `grok.*` / `agent.*` ).
- **Extension bridge.** `p9ext:grok` is exposed only to extension ids that match `/(^|-)panel9-grokbot$/` ; ops are whitelisted ( `state/send/takeControl/markSeen/setSettings/test/retry/openAgent` ), fields clamped, the webhook key is write-only. `panel9-grokbot` (MV3, `storage` only, CSP `default-src 'none'` ) is a popup with Messages and Webhook tabs, the "Grok Bot online" pill, "Request Grok take control" and a Test POST button.
- **Auto-escalation.** When the model is unavailable/over quota or its tool calls fail 3 times, the Agent sends an `escalation` message to Grok Bot ( `reason` in the payload) if "Ask Grok Bot automatically when I get stuck" is on; limited to 10 per hour and de-duplicated within 60 s.
- **Tests.** `test:agentdb` , `test:agentcore` (mock Gemini), `test:agenthost` , `test:agent` (real pane + webhook server + real p9ctl, 60 checks), `test:grokext` (22), `test:agentshots` (theme screenshots), `test:launcher` . All in `test:modules` .

## 2.3.8 — Windows on ARM64: native helper, machine-CPU detection ( `src/hostarch.js` , `src/directhelper.js` )

- **Problem (found on a Snapdragon X Elite PC by the Extend bot, `/workspace/shared/expressvpn-ROOTCAUSE-arm64.md` ):** Panel 9 is an x64 Electron app and runs under emulation on Windows on ARM. The bundled x64 Go helper `Panel9 Direct.exe` started and listened, then every outgoing dial stalled (and the process could not even be killed), so Outside VPN timed out. The native arm64 build of the same source works.
- **Fix:** the build ships both `Panel9 Direct.exe` (amd64) and `Panel9 Direct arm64.exe` ( `tools/build-helper.js` , `extraResources` ). `hostArch()` reports the CPU of the MACHINE (env `PROCESSOR_ARCHITEW6432` / `PROCESSOR_ARCHITECTURE` , then `reg query HKLM\...\Session Manager\Environment` , absolute path to `reg.exe` , then `process.arch` ), because `process.arch` says x64 in an emulated app. `DirectHelper.sourceBin()` picks the arm64 exe there (x64 only if the bundle has no arm64 file).
- **Install step:** whichever architecture is chosen is always copied to `userData\helper\Panel9 Direct.exe` (the NAME never changes: ExpressVPN's split-tunnel entry is path based). Copy only when the file differs (new helper version or other architecture); a native arm64 helper already there is never replaced by a bundled x64 one ( `peArch()` reads the PE machine type); a locked/running copy is renamed aside ( `.old-<ms>` ) and the new one put in place, leftovers are swept at the next start; if nothing works the error says what to do. `status()` shows `arch` and `hostArch` .
- **Tests:** `npm run test:hostarch` ( `test/hostarch-checks.js` , plain node, fake PE files; lock simulation).
- **Native arm64 Panel 9 (not shipped, future work):** `electron-builder --win dir --arm64` builds cleanly on the box (PE machine 0xAA64; no native node modules: SQLite is Electron's built-in `node:sqlite` , so there is no better-sqlite3 arch issue), but it cannot be run or tested here (no ARM Windows) and the store/update contract has no architecture dimension yet ( `platform=windows|windows-setup` only). To ship it: add `arch` to the store release/latest contract, publish `windows-arm64` / `windows-setup-arm64` , make `updater.js` request the arch from `hostArch()` , and test on a real ARM64 PC first. Until then the emulated x64 app plus the native helper is the supported setup.

## 2.3.8 — Gemini model discovery ( `src/agent/models.js` , `src/agent/gemini.js` )

- No model name is hard-coded as the only choice. `Gemini.refreshModels()` pages through `GET /v1beta/models?pageSize=1000` (key in the `x-goog-api-key` header, never in a URL or log) and `models.pick()` ranks what the key can use per role ( `fast` = flash-lite, `deep` = flash, `embed` = embedding): stable over preview/experimental, newer version, plain over dated variants, `-latest` aliases last; TTS / image / live / audio / gemma / embedding-in-chat etc. are excluded. Picks are cached in kv `models.cache` ( `{picks, at, override}` , 24 h TTL); the user can override a role in Agent → Settings → Models. `DEFAULTS` are only a last resort.

- `_withModel(role, explicit, fn)` wraps every call: on a "gone" error (`isModelGone`: 404 / 400 / 403 / 410 whose text says no longer available / not found / not supported for generateContent ...) the model is added to the session `bad` set, the list is refreshed and the call is retried on the next best (up to 4 distinct models). `_post` maps these to `model_gone` before the key mapping, so a 403/404 retirement is never mistaken for a bad key.
- Request shape follows the model generation: Gemini 3+ never gets `temperature` / `top_p` / `top_k`; `thinkingConfig.thinkingLevel` is `medium` when tools are present on 3.x (2.5 uses `thinkingBudget: 0`); `thoughtSignature` on parts and `id` on function calls/responses are preserved; `thought:true` parts are hidden; a 400 mentioning thinking/temperature is retried once without them.
- Embeddings: `gemini-embedding-001` uses `batchEmbedContents`; embedding-2+ uses one `embedContent` per text (4 in parallel, no taskType). Different embedding models have incompatible vector spaces, so `db.embedModel()` records which model produced the vectors and `resetVecs()` + background re-embedding run when the chosen model changes.
- Glue ops: `agent.models`, `agent.refreshModels`, `agent.setModels`; `agent.setKey` clears picks and refreshes. Tests: `npm run test:geminimodels` (plain node, strict mock incl. retirement and pagination) and the e2e picker checks.

## 2.3.8 — VPN guard (`src/vpnguard.js`; from `expressvpn-core.patch`, review notes in `/workspace/shared/expressvpn-core-NOTES.md`)

- Folded in as delivered (10 files): `vpnguard.js` (pure verdict / precedence / watchdog logic), `main.js` VPN block (`verifyBypass`, `routeChanged`, `reapplyWebRtc`, `helper-selftest`, `split-fix`, `config.dns`), `directhelper.js` (watchdog with abandon-if-unkillable, restart cool-down, `egress()`, `flush()`, stand-alone copy preference), `expressvpn.js` (`diag`, `ensureSplitTunnel`), `extbridge.js`, `control.js` + `p9ctl vpn diagnose`, tests `vpnguard-checks.js` / `vpn-checks.js`. Two integration fixes of ours: `vpn-checks` verdict regex widened for extension 1.0.1 wording, and the stand-alone helper copy is only preferred when its architecture is right for the machine (a wrong-arch copy on ARM is ignored).
- Helper 1.1.1 (x64 + arm64) is bundled; `helper/p9direct/*.go` (Extend's source) is committed.

## 2.3.8 — Find / Replace and Lounge search (`src/pages/findbar.js`, `src/textsearch.js`, `src/lounge.js grep`)

- One find UI, two engines. `L.findExt()` is the CodeMirror extension used by `cmBase` (Code, Text, XML): a custom `search({top:true, createPanel})` panel (only ONE `search()` config may exist or CodeMirror throws a merge conflict, so `cmBase` itself was changed) with Case / Word / Regex pills, "n of N" counter (capped at 10 000), Replace / Replace all, optional "All files" scope, `Mod-f`, `Mod-h`, `F3`, `Mod-g`, `Esc` (high precedence). `L.findUI(adapter)` is the engine-independent bar (also used by the CSV grid with its own cell engine). Options persist in localStorage `find.opts`.
- "All files": the Code editor passes `tabs()` / `goto()`; matches are computed with `SearchQuery.getCursor(doc)` on every tab's document (the active tab's live doc, so unsaved edits count), capped at 300 listed / 5 000 counted, one row per line.
- `?fq=&fo=cwr&ln=` on an editor URL opens the bar on that text and selects the match on that line (`L.urlFind`). `lounge.open {path, find:{q, regex, caseSensitive, wholeWord, line}}` builds it.
- Lounge search: `Lounge.grep(q, {regex, caseSensitive, wholeWord, content, names, perFile, limit, isText})` (pure Node, `textsearch.buildRegex/searchText`; whole word uses Unicode lookarounds; text files up to 1 MB, binary skipped, invalid patterns give a readable error, empty matches never loop). api9 `lounge.grep`; `lounge.search` (page and `p9ctl`) switches to it when options or `hits` are given.
- Main-process shortcut: Ctrl+H is passed to the page on `panel9://(text|code|csv|xml)/` (it opens History everywhere else).
- Tests: `npm run test:find` (`test/find-checks.js`: matcher + grep unit checks, real editors with real key events, CSV, Lounge page, open-at-line).

## 2.3.8 — Agent popup (`src/agent/glue.js` `popup` / `ui`, `src/pages/agent.*`, `src/main.js`)

- `ctx.agentPopup = {view, w, h, anchor, closing, keep}` is a transparent `WebContentsView` over the page content, under the toolbar (`y = chromeHeight`), right-aligned to the `#agent` button (rect read from the UI view, refreshed on layout; re-added to the child list on every layout so it stays on top). `panel9://agent/?popup=1` draws the

matte card itself (14 px transparent margin for the shadow) with CSS `popin` / `popout` animations; close = page `.out` class, view removed after 170 ms (instant in tests/when replaced).

- Settings (per profile, `store.js`): `agentMode` (`popup | pane`, default popup), `agentPinned`, `agentToolbar`, `agentPopupW/H`. `ui` is the single entry (`open`, `close`, `toggle`, `setMode`, `setPinned`, `state`); toolbar button, Ctrl+Shift+Space, the menu, Grok Bot and the control relay all go through `agentG.toggle/ui.open`. `setMode` carries the half-typed message (`?draft=`); chat state is the profile database, so both views show the same conversation (events go to every agent view via `targets()`).
- Outside click: the popup `webContents` `blur` (140 ms debounce, ignored while resizing, pinned, a native menu is open, or it was opened in the background and not yet used); the click on the toolbar button that caused the blur does not reopen it (300 ms guard). Esc closes unless pinned.
- Resize: three grips (left, bottom, corner) in the page, `screenX/Y` deltas (the view moves while dragging), `agent.popupResize` clamps 340–760 × 320–window, final size saved. Toolbar: `state.agent {show, open, mode}` drives the button (`hidden`, `.on`); right-click → `agentBtnCtx` menu (`agent-button`).
- Tests: `npm run test:agentpopup` (`test/agent-popup-checks.js`, real window, real key events).

## 2.3.10 — Bean ExpressVPN bridge (`src/extbridge.js` `vpn` op, `main.js` `beanVpn`, `extensions.js peek`; patch from YT Likes, `/workspace/panel9-bean/panel9-bean-vpn-bridge.patch`)

- `p9ext:vpn` routes by extension id: the Bean id (`BEAN_ID_RE`) goes ONLY to `beanVpn` with `BEAN_VPN_OPS = regions|state|connect` (location ≤ 120 chars, no CR/LF/NUL, booleans strict); it never reaches the ExpressVPN extension's richer `vpnCall`. Other ids keep their existing rules. `connect` accepts only `''` / `smart` or an id from the CLI's own region list (hardened in 2.3.10: exactly `smart`, not any `smart…` prefix), and returns `{ok, accepted}` immediately (poll `state`).
- `state.vpn.exitIp` is looked up through a plain session (`p9-bean-vpn-plain`, no proxy, connections closed first), cached 3 s per (state, location, gen); `routes[]` lists each container as `vpn | bypass | proxy`.
- Background runner (`beanRun`): a hidden muted `WebContentsView` of Bean's popup page (`?runner=1`) in the first non-private window, only while `p9bn.api.enabled` has a token (read with `extMgr.peek`, a main-process-only in-memory read; the token is never sent anywhere). Test seam `P9_NO_BEAN_RUNNER`.
- Tests: `npm run test:beanvpn` (`test/bean-vpn-checks.js`, mock expressvpnctl); part of `test:modules`.

## 2.3.11 — Outside-VPN stability, owned helper, Lounge file viewers

- **Route key** (`vpnguard.routeKey`) = `state|location`; the public address is deliberately not part of it. `onRouteChanged` merges evaluations inside `routeGapMs()` (10 s; env `P9_VPN_ROUTE_GAP_MS` for tests) and expires (not wipes) the exit cache. `verifyBypass` stale re-checks: 2 per 60 s (`allowRetry`).
- **Helper** (`directhelper.js`): restarts ask for `lastPort`; the watchdog needs 3 missed pings (`Health.limit`), `noteResume()` clears it; `killNow()` on quit / exit. During a short outage (`graceMs()` 3 s; env `P9_DIRECT_GRACE_MS`) the sessions keep pointing at the dead port (still fail-closed) instead of being re-pointed. `applySessionProxy` returns whether the config changed; `reapplyProxies({force})`.
- **Ownership** (`helperreap.js`): `isOwnedImage` = direct child of an owned dir AND basename `/^panel9 direct.*/i`. `reap()` (injectable list/kill) ends only those. `psStopHelpers()` is the same match as PowerShell for the updater wrapper; `build/installer.nsh customInit` does the same by path in NSIS.
- **UI churn**: `pushState` de-dupes by JSON signature; `layout()` uses `setBoundsIf/setVisibleIf/setRadiusIf`; `viewguard.repaint(view, {nudge})`, `uniformBitmap` over the whole viewport.
- **Lounge viewers**: `P9Files.detect(name, bytes, mime, {zipNames, zipMime})` (magic bytes, BOMs, text vs binary, delimiter sniffing) decides the editor; `loungeglue.detectFor/editorForLounge` feed it from `lounge.open`, `openLocalFile`, `openUrlInEditor`. New pages `sheet` (SheetJS CE + `sheetcalc.js` formula engine + `xlsxstyle.js` / `odsstyle.js` for fonts / fills / widths that SheetJS CE does not return) and `view` (images, media, docx via mammoth into a sanitised DOM, `zip.js` listing, "can't preview" card). `markdown.js` and `jsontree.js` feed the code editor's preview iframe. Vendor libraries are served from `/vendor/` (licences in `src/vendor/LICENSES-2.3.11.txt`).
- **Tests**: `test:helperown`, `test:filetypes` (real files in `test/samples`, made by `make-samples.py`), `test:files` (Lounge + spreadsheet + viewers in Electron), `test:lndetails` (details view at 100/125/150 % in both themes; screenshots in `shots/lounge-details-*.png`).

## 2.3.13 — who really pressed it (trust.js, confirmhost.js)

- `src/trust.js` keeps two per-webContents stamps: **user** (a trusted click / key reached a main-owned preload: `ext-bridge-preload.js` for extension popups, `confirm/preload.js` for the card) and **auto** (stamped by every automated input path: `Control._viaCdp` and `ctlsurface._act`). `verified(wc)` = a user stamp in the last 4 s and no auto stamp within 8 s of it. CDP / `sendInputEvent` produce trusted events too, which is why automation has to stamp itself instead of the page being trusted by `isTrusted` alone.
- Webhook `origin`: `GrokChannel.send({origin})` takes it only from main-process call sites (`glue.js` `trustOrigin()` for the extension, `'echo'` for Echo / the Agent page); the extension bridge never forwards one from the page. Default `system`.
- Confirmation: `GrokChannel.confirmRequest / confirmDecide / confirmStatus` (state in `grok_msgs.meta.confirm`, HMAC key in the Agent db, encrypted), `confirmhost.js` owns a protected `WebContentsView` (`trust.protect`) laid over the Grok extension window or the browser window; `control` refuses to target protected webContents. The only path to `confirmDecide(..., {origin:'user'})` is `ipcMain` `'p9confirm:decide'` from that view, requiring a trusted click in its preload.
- `dialoghost.list/answer` back `p9ctl dialogs` / `dialog`; `accounts.recheck` backs `p9ctl signstatus check`.

## Echo skills (2.3.14)

`src/agent/skills.js` (pure, no Electron): `SkillRegistry` (bundled `src/agent/playbooks`, `playbooks/synced` copied at build by `tools/sync-echo-skills.js`, `<userData>/echo-skills`, Lounge folder “Echo skills”, dev slot `/workspace/panel9-echo-skills`), `PROBE_JS` (one-round-trip page snapshot), `classifyGoogle` (22 page states), `runGoogleSignin` (state machine with injected `io`: probe / goto / fill / clickText / clickSel / waitChange), TOTP (RFC 6238) and notes parsing (`totp:`, `recovery:`), redaction. `Control.skillIO(ctx, t)` implements `io` with focus-free CDP input (type is verified by length, never read back). `agenthost.js` hosts `list_echo_skills` / `read_echo_skill` / `run_echo_skill`: picks the container and e-mail, reads the login from the password store (never returned), opens ONE tab flagged `t.p9resign` (main.js overflow functions skip it: a re-sign-in never creates a container), asks the user through `o.confirm` for captcha / phone / SMS, confirms success with `accounts.recheck` (signstatus). `local.js` maps “sign in to google …” to the skill without a model. Tests: `test/echo-skills-checks.js` (node, mocked Google branches), `test/echo-skills-e2e-checks.js` (real app + mock Google).

## Echo parity with p9ctl (2.3.16)

Every p9ctl command has an Echo tool (matrix in `test/echo-parity-checks.js`; the deliberate exceptions are `eval` and raw `html`). Tools live in `src/agent/tools.js` (registry, `confirm`, `SECURITY_SETTING`), handlers in `src/agent/agenthost.js` (through `d.control.runLocal` for page work: `LOCAL_CMDS` now also has `watch` and `add-account`; through `hostDeps` for `signstatus` / `procs` / `dialogs` / `dialogAnswer` / `focusOrOpen`), `run_steps` in `Agent._exec` (each step goes through `runTool`, so validation and the confirm card apply per step). Risky tools ask via `needsConfirm`; delete/sign-out containers additionally show Panel 9's own `confirmBox`.

## Light sessions (2.3.16)

`ctx.sessionFor(cid, { light: true })` creates the container's session with proxy + ad-block but without extensions (`sesExtPending`); `attachPendingExt` loads them on the first full `sessionFor`. `accounts.recheck` (signstatus, Echo `check_signin_status`, the google-signin confirm step) uses light sessions, so checking 19 containers does not start 19 sets of extension renderers.

## Idle release + scoped extensions (2.3.17)

`src/main.js`: `sesBusyAt` (WeakMap session → last use), `sesInUse(ctx, ses, cid)` (any non-host WebContents of that session, sign-in tabs, transfers), `releaseSessionExt(ses)` (flush storage, `extMgr.detachSession`, `sesExtPending.add`, `sesReleased`), `releaseIdleSweep({force})` every 30 s (never `default`, private windows, sessions still loading), setting `releaseIdleMin` (default 3 min; tests set `P9_TEST_RELEASE_MS` to use a few seconds). A released session is just a light session: the next full `ctx.sessionFor(cid)` calls `attachPendingExt`, which re-attaches with `{scoped}`. `src/extensions.js`: `scopeOfManifest` ('popup' = no content scripts/background/host permissions, only storage/clipboard/notifications), `attachSession(ses, {scoped})` skips popup-scoped extensions, `ensureLoaded(ses, id)` loads one on demand (called by

`openExtensionPopup`), `isReleasable`, `loadedIds`, `hostWebContents`. Anything that only needs cookies/network must use `sessionFor(cid, { light: true })` or it re-attaches extensions. `procsReport()` returns `byContainer`; p9ctl prints it; `p9ctl procs release / perf.releaseNow` / Echo `release_idle_containers` force a sweep. Test: `test/release-idle-checks.js`. Helper 1.1.2 (`helper/p9direct/server.go`): `dialHost0` records every attempt, classifies by IPv4 first, skips IPv6 without a global IPv6 address; `/p9direct/dialtest`. Helper 1.1.3: on a machine with no IPv6, a CONNECT/SOCKS target that is an IPv6 literal (name looked up in the resolver's ip->name memory, `resolver.nameFor`) or a name whose answers are all IPv6 is re-resolved over IPv4 with an explicit A query (`resolver.lookup4`, DoH then OS fallback) and dialled on IPv4. An IPv6-only destination that still fails is the ONLY thing the helper records: host or IPv6 literal (+ known name), kind, count, in `/p9direct/status` -> `dials.ipv6OnlyFailures` and one stderr line per host per minute. No other destination is ever logged.

Helper 1.1.5 / Panel 9 2.3.20: `session.fetch` has no proxy-login path (a tab answers through app `login`, `net.request` through its own `login` event), so a light session that never logged in to the helper got the 407 and failed with ERR_TUNNEL_CONNECTION_FAILED. `ensureProxyAuth` (main.js) logs every Outside-VPN session in with a `net.request` to `http://p9direct/p9direct/health` (answered by the helper, no outside traffic) when the proxy is applied, in `ctx.sessionSettled` and after a helper restart (`tokenGen`). `/p9direct/status` has `conn` counters (connect/forward/socks5/control/noCreds407/badCreds407/rejected); `signstatus check` shows `proxy` (resolved rules, applied config, helper login) for an unreachable container.

## 2.3.21 — YouTube Focus lean page: session-level request filter (`src/ytf-lean.js`, `src/reqchain.js`)

Why: YouTube Focus 1.4.0 "Comments only" is meant to *load* a weightless YouTube. In Chrome the extension does that with a declarativeNetRequest ruleset (`rules-lean.json`) toggled by its service worker; Electron's extension platform has neither, and the page world cannot stop the parser-inserted `<script src=base.js>` tags, so Panel 9 does it in the main process.

- `src/reqchain.js` — **one** `ses.webRequest.onBeforeRequest` listener per session (Electron keeps only the last one registered). `add(ses, name, fn)` / `remove` / `has`; filters run in registration order, `fn(details) === true` cancels, an exception is ignored. `adblock.js` (`_verdict`) and `ytf-lean.js` are filters in it (`main.js` attaches the ad blocker first, then the lean filter, in `Context.sessionFor`).
- `src/ytf-lean.js` — `leanCancel(url, resourceType)` (pure: the block list; `mainFrame` is never cancelled) and `YtfLean` (`on()` = the YouTube Focus extension is enabled and `peek('commentsOnly') === true` — `extMgr.peek` mirrors `chrome.storage.local`, the ids tried are `yt-focus`, `local-yt-focus`, `local-youtube-focus`; `_fromWatch(d)` = the requesting webContents' URL (or, for the service worker which has no webContents, the referrer) is a youtube.com watch URL or `/sw.js`). `stats`, `recent` (last 50 cancelled URLs). Not cancelled, ever: `youtubei/v1/next`, `youtubei/v1/comment/*`, browse, ytcfg/app scripts, fonts/icons, `/signin`, `/getAccountSwitcherEndpoint`, accounts.google.com, other sites, non-watch YouTube pages.
- Service worker: YouTube's `/sw.js` pre-caches the player with its own XHRs and serves base.js from its cache (invisible to webRequest). The filter cancels `/sw.js` and the worker's player requests; `_poll()` (400 ms, in-memory mirror) wipes `serviceworkers` + `cachestorage` of `https://www.youtube.com` in every attached session when the switch goes ON, and `attach` does it for a session attached while ON. Never wipe inside the request listener (freezes the main process).
- Test hooks: `api.ytfLean`; tests `test/ytf-lean-checks.js` (node), live check `tests/live/p9-live.js` in the yt-focus-extension repo (real YouTube, box test copy). `npm run test:ytf` still exercises the extension against real youtube.com.
- Gotchas: a second `onBeforeRequest` registration anywhere else in `main.js` would silently replace the chain — register through `reqchain`. Panel 9 never cancels anything unless the extension's switch is ON.

## 2.3.22 — Channel switch: main frame vs fetch investigation, `src/navtrace.js`, lean listener only while ON

Bean (channel switch through YouTube's `signinUrl`) reported: after the switch a top-level navigation to the watch page renders the OLD channel (ytcfg `DELEGATED_SESSION_ID` and masthead) while an in-page `fetch()` of the same URL renders the NEW one. **Audit result: Panel 9 treats main-frame and subresource requests identically for cookies, headers and caching in container sessions.** The only `webRequest` handlers are `onBeforeSendHeaders` (`main.js setupSession`: Sec-CH-UA / UA hints, mobile profile; never Cookie) and the `reqchain` `onBeforeRequest` (ad blocker + lean filter; both skip `mainFrame`). No `onHeadersReceived`, no https `protocol.handle/intercept`, no cookie writes, no cache mode, no per-webContents cookie

cache; the Outside-VPN helper is a plain CONNECT tunnel. `test/switch-nav-checks.js` reproduces the switch on a mock (302 that sets cookies) in plain / ad blocker / lean / mobile / proxy configurations: the main-frame request and the fetch send the same cookie names, counts and value hashes and both see the NEW channel. The remaining suspect is YouTube's own handling of `Sec-Fetch-Dest: document` vs `empty` (or the signin interstitial's timing); it needs real traffic, hence the trace below.

- `src/navtrace.js` — **opt-in observer**, off by default. Enabled at app start when env `P9_NAVTRACE=1` or the file `<userData>\navtrace.on` exists ( `app.getPath('userData')` = `%APPDATA%\Panel 9` , one folder for all profiles; a restart is needed because the switch is read once at startup). `NavTrace.attach(ses, label)` (called from `setupSession` for every container / profile / private session) adds `onSendHeaders` , `onBeforeRedirect` , `onResponseStarted` observers (no callback: cannot change or delay anything; none of them is used elsewhere, so no listener is replaced). Records only for google.com / youtube.com / googlevideo.com / ytimg.com hosts: request side = Cookie NAMES in sent order (duplicates counted) + per-run salted 6-hex hash of each value, Sec-Fetch-Dest/Mode/Site, Accept (24 chars), Referer present, UA hash, Sec-CH-UA present, Accept-Language, X-Goog-AuthUser/PageId present; response/redirect side = status, Set-Cookie NAMES with Domain/Path/SameSite/Partitioned/expiry flags and hashed value, cache-control, vary, `fromCache` . Label = `<profileId>/<containerId>` . Never a cookie value, never a query string. Output: `<userData>\navtrace.log` (JSON lines, appended across runs, at most 3 MB written per run) and the in-memory ring `api.navtrace.dump()` (400 entries).
- `src/ytf-lean.js` — `_sync(ses)` : the lean filter's session listener is registered only while Comments only is ON (2.3.21 registered it on every session at attach time); `attach` and the 400 ms `_poll` keep it in step with the switch. OFF ⇒ the session has no `onBeforeRequest` unless the ad blocker is on (2.3.20 network path).
- Tests: `npm run test:navtrace` (node), `npm run test:switchnav` (Electron, xvfb), `test/ytf-lean-checks.js` (+1 check); both new scripts are in `test:modules` .

## 2.3.23 — remote-control gaps, honest ping, calm Accounts

- **Where the version comes from**: `ping` (src/ctlv23.js) reports `VERSION` (package.json via main.js) + `os.hostname()` ; p9ctl has its own `P9CTL_VERSION` . 2.3.22 and earlier did not report any version.
- **src/ctlv23.js** (installed on `Control.prototype` , `_v23Cmds(base)` ; `get cmds()` in control.js merges it over `_surfaceCmds()` ): ping, tabs `--opened-by` , close (id required), dialog.list/answer, screenshot `--window` ( `captureWindow` : composite of the visible views in z-order, protected surfaces skipped), containers.remove|signout|rename|recheck|identity, notify, agent.ask/result/cancel/pending (asks registry), eval ( `__safe` serializer, isolated world first then main world via CDP), whoami. READ_ONLY list in control.js knows the new read-only commands.
- **Safety**: remote dialog answers carry `remote:true` ; `dialoghost.answer` refuses a destructive accept without a trusted user event ( `trust.verified` ); remote destructive container actions run inside `dialogHost.withOrigin({origin:'remote'})` , show the normal confirmation (marked "asked by Grok Bot") and return `awaiting-user` . Main container is refused for remove / rename. `protect()` ed surfaces (Echo Confirm card) are never captured.
- **reauth** state: `accounts.statusOf` → `in|out|unknown|reauth` ; `recheck` = cookies + ListAccounts + deep YouTube redirect trace ( `accounts.trace` , `net.request` with manual redirects; `ses.fetch` hides the final URL in Electron). `src/googlechallenge.js` classifies Google challenge URLs; main.js `flagChallenge` flags / clears tabs and the container.
- **src/ytidentity.js**: which channel / account a container is on (ytcfg + `account/accounts_list` , SAPISIDHASH computed in memory, cookie value never returned).
- **Relay**: `control.js` heartbeat ( `caps.hb` ), backoff, `networkChanged` , 4001 hold-off, `status()` ; relay.py additions are additive ( `ping/pong` , `caps` , `undelivered` , presence, `inst` ).
- **Echo ask** : `Agent.ask` bounded queue ( `queueMs` ), abort, `onStart` ; `glue.controlApi.agent.ask` passes them through.
- **Accounts UI**: `src/pages/accounts.css` + `accounts.js` (see CHANGELOG-2.3.23). Menus: `containerMenuRow()` in main.js, `view.st` whitelisted in `menuhost.serialize` , `ROOMY_KINDS` ( `new-tab` , `auth-container` , `container` ) + sticky filter ( `addFilter` ) in `src/menu/menu.js` , styles at the end of `menu.css` .
- **Tests**: test/ctl-v23-unit-checks.js (plain node), ctl-v23-checks.js and ctl-reconnect-checks.js (real relay.py + real p9ctl via test/ctl-harness.js). test/accounts-shots.js is a before / after screenshot generator, not a pass / fail test.
- **Background tabs (Bean `background:true` )**: `createTab(..., {background:true})` → `t.keepAlive` , `webPreferences.backgroundThrottling:false` , hidden view kept at the real tab rectangle by `layout()` , a ~120 ms 1-px "blip" at creation so Chromium sizes the page ( `BG_BLIP_MS` ), `endKeepAlive` on activation or after `BG_KEEP_MS` ;

`sleepBlocker` returns `background`; `tabs` info has `background:true`. Why not the 1-px-sliver-forever or covered-view approaches: see CHANGELOG-2.3.23. Probe scripts: /workspace/p9-bgprobe (mock + real YouTube).

## 2.3.24 — relay-drop grace, `tabs --opened-by` hint, lazy session restore, bounded navtrace

- `src/control.js`: `networkChanged` ignores an event within `Control.NET_GRACE_MS` (2.5 s, env `P9_CTL_NET_GRACE_MS`) of the socket being opened (`connectAt`) unless it is a resume/unlock; the heartbeat is still only enforced after the relay announced `caps`/`pong` (an old relay never causes a false drop). Tests: `test/ctl-reconnect-checks.js`, `test/ctl-oldrelay-checks.js` (runs against `test/fixtures/relay-2.3.22.py`, the deployed relay; the harness takes `P9_TEST_RELAY`).
- `tools/control/p9ctl`: an empty `tabs --opened-by X` makes one extra read-only `tabs` call and prints which owners exist.
- `src/main.js` session: `restoreWindowTabs` creates non-active http(s) tabs with `asleep:` (lazy; `P9_RESTORE_EAGER=1` = old); `focusOrOpen` sends a profile that was not started in this run through `startProfile` (restore); `quitForUpdate` saves with `reason:'update'` first; `sessionSaveNow` keeps `session.prev.json` when a >= 3-tab session shrinks to <= 1 tab. Test: `test/session-restore-checks.js` + `session-restore-child-checks.js` (second process on a copy of the user data, two profiles, a container).
- `src/navtrace.js`: batched async writes, 2 MB per-run budget that stops the observer (`stopped` marker), rotation to `.old` above 2 MB at start; `navtrace.on` is one-shot (deleted when read). Test: `test/navtrace-checks.js`.
- **Bean background tabs, corrected (supersedes the 2.3.23 "hidden" design)** — `src/main.js` `layout()`/`bgMode()`/`bgDisabled()`: a `keepAlive` tab that is not active is VISIBLE at its real size at `{x: 1 - w, y: 1 - h}` so only its bottom-right pixel is inside the window. Hidden views paint at ~1 fps (and not at all after a renderer swap), views covered by the active tab are not painted; a visible one is. `P9_BG_MODE=hidden` keeps the old path (plus a blip after every `did-navigate`/`dom-ready`); `P9_BG_TABS=0`/`bgtabs.off` = no keep-alive at all. `ctlv23` window shots skip parked views. Live check: `test/bg-ytf-live.js`.

## 2.3.25 - Background tabs switch

`appPrefs.bgTabs` (default true, `appprefs.json`) + `bgState()` -> `{on, setting, forced:''|'env'|'file'}`; `bgDisabled() = !bgState().on` (still the single check in `createTab`). Overrides: env `P9_BG_TABS=0` ('env'), `<userData>/bgtabs.off` ('file'). `bgTabsSet(v)` saves, and when the effective state is off calls `endKeepAlive` on every keep-alive tab + `layout`. Page handler `bgtabs` (Settings row `#set-bgtabs`, disabled + note when forced), control command `bgtabs` (`d.bgTabs`) and `p9ctl bgtabs|settings bgtabs`; also `info.bgTabs`, ping `bgTabs`, `openTab.background`.

### 2.3.25 - control link on the Direct route

`src/wstunnel.js` (`TunnelWebSocket`: WebSocket over HTTP CONNECT + optional TLS; same surface as the standard WebSocket Control uses, plus `ping()` and a `frame` event). `Control` picks a route per connect: `_directProxy()` (deps `d.direct = {enabled, canStart, ensure, proxy, state}`; main.js wires it to `DirectHelper`; enabled unless `P9_CTL_DIRECT=0`, and in tests only with `P9_CTL_DIRECT=1`). `connect()` waits (<= DIRECT_START_MS, once per 30 s) for `ensure()`; `_connectNow()` opens a `TunnelWebSocket` (direct) or the global WebSocket (normal). Direct attempt that never opens -> `_directFail()` (cool-down DIRECT_BAD_MS, retry in 250 ms on the normal route); `directAvailable()` (helper `change` -> up) clears it. `networkChanged()` on a Direct socket only probes (WebSocket ping + app ping) unless the event is a resume/unlock. `_hbStart` pings Direct sockets with WebSocket pings and enforces 2.5 x HB_MS once a pong was seen (works with the 2.3.22 relay). Route state: `routeInfo()` in `status()`, ping `control.route`, control-log.

## 2.3.26 - Mobile control (`src/mobile.js`, `tools/mobile/`)

- **Transport**: the phone is one more client of the control relay room (`wss://.../control?room=<hash of token>&role=client`). Messages are sealed `m1.<kid>.<AES-256-GCM base64>` / pairing `p1.<...>` strings the relay forwards untouched (it drops binary frames and caps 40 msg/s, 8 MB: frames are JPEG base64 inside the sealed envelope, about 1.8x the JPEG; the phone queue paces itself to one message per 38 ms and coalesces scroll / hover / view.set). `Control._onMessage` hands `m1.`/`p1.` to `MobileHub`; Grok Bot traffic is unchanged. `Control.active()` subtracts live phones so a phone never lights the "Grok Bot is working" banner (`mobilescope` AsyncLocalStorage also silences `_grokPresence`/`_noteStep` for phone work).

- **Capture** ( `mobilecapture.js` ): `composite()` captures each visible view with `capturePage` (never focuses), blends them in z-order into one BGRA buffer at a width chosen by the phone, `encode()` crops (reporting the crop as cut) and JPEG-encodes; popup / extension windows ("extras") and menus float above; `PRIVATE_PAGE` views are skipped. `hitTest()` finds the view under a window point. Extension view = a phone-only hidden BaseWindow ( `openExtensionPopup(..., {phone:true})` , `show:false` , `setContentSize` , `setZoomFactor` ), captured with `capturePage` of its web contents.
- **Input** is addressed as fractions of the picture the PC last sent: `_resolve` -> window px -> the view -> CSS px (divided by page zoom) with no rounding, then the same `_mouse` / `_insert` / `_press` the p9ctl commands use (CDP `Input.dispatch*` , no OS focus). Ops of one device are serialized (order preserved). Edit detection: `EDIT_JS` in an isolated world lists visible fields + the focused one (as window fractions) every 200-450 ms on the relevant surfaces.
- **Security**: pairing code -> PBKDF2-SHA256 (100k, salt = room) -> pairing key; the phone's request carries a fresh device key sealed with it; devices persisted ( `mobile.json` , 0600). Unknown / revoked kid gets one unsigned `unpaired` hint (rate limited). Risky ops ( `RISKY` table) are two-step and server enforced; `mobilescope.confirmed()` tells the PC dialog layer the owner already confirmed.
- **PWA**: no framework; `app.js` (Link, view model, gesture engine, keyboard proxy `#kin` , sheets), tokens + icons copied from Panel 9 ( `sync-assets.js` ), stale-while-revalidate service worker for the shell only. Served by the appstore server at `/m/` ( `tools/mobile/server-patch/mobile_bp.py` , strict CSP).